

Comparative Analysis of Crude Oil and Natural Gas Prices Using Data Analytics and Technical Indicators (2023 – Feb 2026)

Project Overview

This project analyses the historical price behaviour of **Crude Oil** and **Natural Gas** from **2023 to 15 February 2026** using Python, SQL, and Power BI. The study combines **data analytics, technical indicators, and real-world global events** to understand price movements, volatility, and market trends in the energy sector.

The project aims to demonstrate how **data-driven insights**, when combined with **macroeconomic and geopolitical news**, can support informed decision-making for traders, analysts, and business stakeholders.

Business Problem Statement

Energy commodity prices are highly volatile due to:

- Geopolitical conflicts
- Supply–demand imbalances
- Weather conditions
- Economic cycles

Understanding **why prices move**, not just **how they move**, is critical for:

- Risk management
- Trading strategies
- Energy sector planning

This project bridges **technical analysis** with **global news events** to explain price behaviour logically.

Objective

- Analyse historical price trends of crude oil and natural gas
- Measure volatility and risk using statistical methods
- Identify market trends using technical indicators (MA, RSI, MACD)
- Detect major trend reversals (Golden Cross & Death Cross)
- Link price movements with **real global events and news**
- To perform structured analysis using SQL queries.
- To visualize insights using Power BI dashboards.

History of Crude Oil and Natural Gas

Crude Oil

- Crude oil has been a primary source of energy since the 19th century.
- It is widely used in transportation, power generation, and the petrochemical industry.
- The price of crude oil is influenced by production levels, geopolitical conflicts, decisions made by oil-producing countries, and global economic conditions.

Natural Gas

- Natural gas is considered a cleaner fossil fuel compared to crude oil.
- It is mainly used for electricity generation, cooking, and industrial purposes.
- Natural gas prices are highly sensitive to seasonal demand, weather conditions, and supply disruptions.

Dataset Description

Data Source: Yahoo Finance

- Crude Oil ticker: CL=F
- Natural Gas ticker: NG=F

Time Period: February 2023 – 15 February 2026 (3 Years)

Frequency: Daily

Column Details

Column Name	Description
Date	Trading date of the commodity price
Open	Price at the beginning of the trading day
High	Highest price recorded during the trading day
Low	Lowest price recorded during the trading day
Close	Price at the end of the trading day
Volume	Total number of contracts traded on that day
Commodity	Type of commodity (Crude Oil / Natural Gas)

Tools & Technologies Used

- **Python:** Pandas, NumPy, Matplotlib, y finance
- **Database:** MySQL
- **Visualization:** Power BI
- **Storage Format:** CSV
- **Indicators:** RSI, MACD, Moving Averages, Volatility Metrics

Data Collection Methodology

Tool Used

- Python (y finance, pandas, numpy)

```
import pandas as pd

import yfinance as yf
# --- Step 1: Function to fetch daily data ---
def fetch_daily(symbol, period="3y"):
    data = yf.Ticker(symbol).history(period=period, interval="1d").reset_index()
    data['Commodity'] = symbol
    data['Date'] = data['Date'].dt.date
    data['Change%'] = ((data['Close'] - data['Open']) / data['Open']) * 100
    return data[['Date', 'Commodity', 'Open', 'High', 'Low', 'Close', 'Volume', 'Change%']]

# --- Step 2: Fetch Crude Oil and Natural Gas ---
daily_crude = fetch_daily("CL=F")
daily_gas = fetch_daily("NG=F")

# --- Step 3: Combine both datasets ---
daily_df = pd.concat([daily_crude, daily_gas]).reset_index(drop=True)

# Optional: sort by Date & Commodity
daily_df = daily_df.sort_values(['Date', 'Commodity']).reset_index(drop=True)

# --- Step 4: Save combined CSV ---
daily_df.to_csv("Crude_NaturalGas_3Years.csv", index=False)
print("Crude & Natural Gas 3-Year Data Saved: Crude_NaturalGas_3Years.csv")

# Optional: preview
print(daily_df.head())
print(daily_df.tail())
```

Explanation

This script automates the **collection and preparation of historical daily data** for crude oil and natural gas over a 3-year period.

Data Fetching:

- Uses the y finance library to download daily open, high, low, close, and volume data.
- Adds a calculated column for **daily percentage change** to measure price movement.

Data Structuring:

- Each dataset is tagged with its commodity name (Crude Oil or Natural Gas).
- Dates are standardized for consistency.

```
Crude & Natural Gas 3-Year Data Saved: Crude_NaturalGas_3Years.csv
      Date Commodity      Open      High      Low      Close  Volume  \
0  2023-02-15      CL=F  78.830002  79.150002  77.250000  78.589996  264486
1  2023-02-15      NG=F   2.603000   2.607000   2.446000   2.471000  147505
2  2023-02-16      CL=F  78.540001  79.540001  77.919998  78.489998  117600
3  2023-02-16      NG=F   2.464000   2.559000   2.358000   2.389000  141459
4  2023-02-17      CL=F  78.070000  78.250000  75.059998  76.339996   93267

      Change%
0 -0.304460
1 -5.071071
2 -0.063666
3 -3.043833
4 -2.215964

      Date Commodity      Open      High      Low      Close  \
1504  2026-02-11      NG=F   3.159000   3.249000   3.055000   3.159000
1505  2026-02-12      CL=F  64.870003  65.099998  62.389999  62.840000
1506  2026-02-12      NG=F   3.222000   3.316000   3.180000   3.217000
1507  2026-02-13      CL=F  62.990002  63.259998  62.139999  62.889999
1508  2026-02-13      NG=F   3.219000   3.278000   3.114000   3.243000

      Volume  Change%
1504  174848   0.000000
1505  369303 -3.129339
1506  222258 -0.155179
1507  224886 -0.158759
1508  113525  0.745571
```

Data Management:

- The combined dataset is sorted by date and commodity.
- Saved as a CSV file (Crude_NaturalGas_3Years.csv) for future analysis.
- Preview commands (head and tail) allow quick inspection of the dataset.

```
df= pd.read_csv("C:\\Users\\HP\\Crude_NaturalGas_3Years.csv")
df.head()
# df.tail()
```

	Date	Commodity	Open	High	Low	Close	Volume	Change%
0	2023-02-15	CL=F	78.830002	79.150002	77.250000	78.589996	264486	-0.304460
1	2023-02-15	NG=F	2.603000	2.607000	2.446000	2.471000	147505	-5.071071
2	2023-02-16	CL=F	78.540001	79.540001	77.919998	78.489998	117600	-0.063666
3	2023-02-16	NG=F	2.464000	2.559000	2.358000	2.389000	141459	-3.043833
4	2023-02-17	CL=F	78.070000	78.250000	75.059998	76.339996	93267	-2.215964

Info

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1509 entries, 0 to 1508
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        1509 non-null   datetime64[ns]
1   Commodity   1509 non-null   object
2   Open        1509 non-null   float64
3   High        1509 non-null   float64
4   Low         1509 non-null   float64
5   Close       1509 non-null   float64
6   Volume      1509 non-null   int64
7   Change%     1509 non-null   float64
8   Year        1509 non-null   int32
dtypes: datetime64[ns](1), float64(5), int32(1), int64(1), object(1)
memory usage: 100.3+ KB
```

Describe

```
df.describe()
```

	Date	Open	High	Low	Close	Volume	Change%	Year
count	1509	1509.000000	1509.000000	1509.000000	1509.000000	1509.000000	1509.000000	1509.000000
mean	2024-08-15 10:18:22.186878720	37.495714	38.050840	36.889891	37.457569	228599.879390	-0.069099	2024.121272
min	2023-02-15 00:00:00	1.533000	1.628000	1.481000	1.575000	0.000000	-12.702264	2023.000000
25%	2023-11-14 00:00:00	2.813000	2.906000	2.747000	2.821000	145225.000000	-1.886555	2023.000000
50%	2024-08-15 00:00:00	6.646000	7.827000	5.900000	7.460000	212196.000000	-0.057177	2024.000000
75%	2025-05-16 00:00:00	71.889999	73.190002	70.639999	71.849998	307568.000000	1.553461	2025.000000
max	2026-02-13 00:00:00	93.779999	95.029999	91.389999	93.680000	735186.000000	24.903916	2026.000000
std	NaN	35.079476	35.543090	34.560517	35.039515	111534.944775	3.050729	0.878119

Preprocessing steps

Convert Data Types:

- Convert dates to datetime format.
- Extract year for year-wise analysis.
- To maintain data integrity and avoid errors in analysis

```
# Convert Date & Extract Year
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df = df.dropna(subset=['Date'])
df['Year'] = df['Date'].dt.year
```

Filter

```
# Filter by Commodity
crude_df = df[df['Commodity'] == 'Crude Oil']
gas_df = df[df['Commodity'] == 'Natural Gas']
```

To separate the dataset into two subsets for focused analysis of each commodity.

Numeric

```
crude_df['High'] = pd.to_numeric(crude_df['High'])
crude_df['Low'] = pd.to_numeric(crude_df['Low'])

gas_df['High'] = pd.to_numeric(gas_df['High'])
gas_df['Low'] = pd.to_numeric(gas_df['Low'])
```

To ensure the values can be used in mathematical operations and statistical analysis.

Replace ticker symbol

```
# Replace ticker symbols with readable commodity names
df["Commodity"] = df["Commodity"].replace({
    "CL=F": "Crude Oil",
    "NG=F": "Natural Gas"
})

# Check unique values
print(df["Commodity"].unique())
```

Ticker symbols like "CL=F" and "NG=F" are technical identifiers.

Readable names improve communication of insights to non-technical audiences.

To separate crude oil and natural gas data for independent analysis

```
crude_df = df[df['Commodity'] == 'Crude Oil']
gas_df = df[df['Commodity'] == 'Natural Gas'].copy()
```

['Crude Oil' 'Natural Gas']								
	Date	Commodity	Open	High	Low	Close	Volume	Change%
0	2023-02-15	Crude Oil	78.830002	79.150002	77.250000	78.589996	264486	-0.304460
1	2023-02-15	Natural Gas	2.603000	2.607000	2.446000	2.471000	147505	-5.071071
3	2023-02-16	Natural Gas	2.464000	2.559000	2.358000	2.389000	141459	-3.043833
2	2023-02-16	Crude Oil	78.540001	79.540001	77.919998	78.489998	117600	-0.063666
5	2023-02-17	Natural Gas	2.391000	2.420000	2.220000	2.275000	133999	-4.851524

Major Global Events Impacting Prices (News-Linked)

Crude Oil Market Events

2023

- **Russia–Ukraine war** continued, disrupting global oil supply routes and increasing uncertainty.
- **OPEC+** announced production cuts, pushing crude prices upward.
- High inflation and aggressive interest rate hikes slowed global demand.

Impact: Increased volatility and frequent price spikes.

2024

- Rising **Middle East geopolitical tensions** increased supply risk premiums.
- Fluctuations in US crude inventories caused short-term volatility.
- Global economic slowdown reduced industrial demand.

Impact: Prices fluctuated within a wide range, reflected in rolling volatility.

2025

- Continued supply control by oil-producing nations.
- Rapid growth in renewable energy raised long-term demand concerns.

- Crude prices showed frequent ups and downs, signalling uncertainty.

Impact: Mixed trends and repeated Golden/Death Cross signals.

2026 (Till Feb 15)

- Balanced global supply and demand led to price stabilisation.
- Early signs of economic recovery supported short-term bullish momentum.

Impact: RSI and MACD showed momentum shift and potential reversals.

Natural Gas Market Events

2023

- Europe faced an energy crisis due to reduced Russian gas exports.
- LNG demand surged, creating high volatility.
- Winter demand caused price spikes.

Impact: RSI overbought signals and volatility spikes.

2024

- Mild winter reduced heating demand.
- Increased US shale gas production lowered prices.

Impact: Lower volatility compared to crude oil.

2025

- Expansion of LNG exports increased global exposure.
- Prices became unstable with sharp spikes.

Impact: Sudden volatility bursts despite overall stability.

2026 (Till Feb 15)

- Extreme price spikes occurred, including a **historical peak in January 2026**.
- RSI and MACD indicated overbought and bearish conditions.

Impact: High-risk market conditions and instability.

Exploratory Data Analysis (EDA)

Daily Price Range

Purpose:

This dataset records the **daily price range** (difference between high and low prices) for crude oil and natural gas. It helps analysts measure **short-term volatility** and understand how much prices fluctuate within a single trading day.

```
# Daily_Range
crude_df['Daily_Range'] = crude_df['High'] - crude_df['Low']
gas_df['Daily_Range'] = gas_df['High'] - gas_df['Low']

# Combine into one DataFrame for easier plotting
crude_df['Commodity'] = 'Crude Oil'
gas_df['Commodity'] = 'Natural Gas'
combined_df = pd.concat([crude_df[['Date', 'Daily_Range', 'Commodity']],
                        gas_df[['Date', 'Daily_Range', 'Commodity']]])
print(combined_df)
```

Output:

	Date	Daily_Range	Commodity
0	2023-02-15	1.900002	Crude Oil
2	2023-02-16	1.620003	Crude Oil
4	2023-02-17	3.190002	Crude Oil
6	2023-02-21	1.820000	Crude Oil
8	2023-02-22	2.750000	Crude Oil
...	...	...	...
1500	2026-02-09	0.154000	Natural Gas
1502	2026-02-10	0.153000	Natural Gas
1504	2026-02-11	0.194000	Natural Gas
1506	2026-02-12	0.136000	Natural Gas
1508	2026-02-13	0.164000	Natural Gas

Explanation

Crude Oil (Feb 2023):

- Daily ranges vary between ~1.6 and ~3.2 USD.
- Larger ranges (e.g., 3.19 USD) indicate strong intraday volatility.
- Smaller ranges (e.g., 1.62 USD) suggest calmer trading sessions.

Natural Gas (Feb 2026):

- Daily ranges are much smaller, between ~0.13 and ~0.19 USD.
- This shows natural gas is relatively stable compared to crude oil.

Year-wise Volatility

Purpose:

To calculate the **annual volatility** of each commodity (Crude Oil and Natural Gas) and compare their risk levels across years.

Formula:

- Volatility Year=std (Price Change of all days in that year)

```
crude_vol = (  
    df[df["Commodity"] == "Crude Oil"]  
    .groupby("Year")["Price_Change"]  
    .std()  
)  
  
gas_vol = (  
    df[df["Commodity"] == "Natural Gas"]  
    .groupby("Year")["Price_Change"]  
    .std()  
)  
  
print(crude_vol)  
print(gas_vol)
```

Output:

```
Year  
2023    1.650600  
2024    1.358132  
2025    1.310528  
2026    1.389050  
Name: Price_Change, dtype: float64  
Year  
2023    0.105546  
2024    0.122222  
2025    0.165308  
2026    0.802282  
Name: Price_Change, dtype: float64
```

Explanation:

- Crude Oil: 2023 → 1.65, 2024 → 1.36 (higher value → more volatility in that year).
- Natural Gas: 2023 → 0.105, 2026 → 0.802 (sudden spike in 2026 indicates high risk or unusual price movement).

Matplot

Purpose:

The chart compares the volatility of crude oil and natural gas prices across the years 2023 to 2026. Volatility measures how much prices fluctuate, helping analysts assess risk and stability in each commodity.

```

import matplotlib.pyplot as plt
import numpy as np

# Years
import numpy as np
import matplotlib.pyplot as plt

years = crude_vol.index.values
x = np.arange(len(years))
bar_width = 0.35

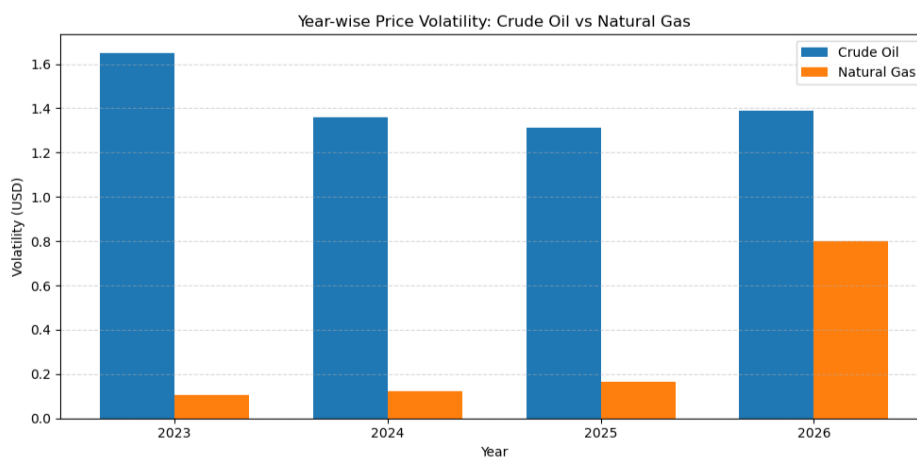
plt.figure(figsize=(10,5))

plt.bar(x - bar_width/2, crude_vol.values, width=bar_width, label="Crude Oil")
plt.bar(x + bar_width/2, gas_vol.values, width=bar_width, label="Natural Gas")

plt.xticks(x, years)
plt.xlabel("Year")
plt.ylabel("Volatility (USD)")
plt.title("Year-wise Price Volatility: Crude Oil vs Natural Gas")
plt.legend()
plt.grid(axis="y", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```

Output:



Explanation:

- Crude Oil (Blue Bars)
 - Volatility remains moderately stable, ranging between approximately 1.4 to 1.6 USD.
 - Indicates consistent but noticeable price fluctuations year-to-year.
- Natural Gas (Orange Bars)
 - Very low volatility during 2023–2025 (around 0.1–0.2 USD).
 - Sharp spike in 2026 (~0.8 USD), indicating increased instability.

Insight

- Crude Oil shows predictable long-term volatility.
- Natural Gas, though stable initially, exhibits sudden risk spikes, especially in 2026

Statistics:

Purpose:

Summarizes annual closing prices of crude oil and natural gas, showing mean, maximum, and minimum values. Helps identify yearly trends, extremes, and compare the two commodities.

```
# Crude Oil
crude_yearly = crude_df.groupby('Year')['Close'].agg(['mean', 'max', 'min']).reset_index()
crude_yearly.columns = ['Year', 'Crude_Mean', 'Crude_Max', 'Crude_Min']

# Natural Gas
gas_yearly = gas_df.groupby('Year')['Close'].agg(['mean', 'max', 'min']).reset_index()
gas_yearly.columns = ['Year', 'Gas_Mean', 'Gas_Max', 'Gas_Min']

print(crude_yearly)
print(gas_yearly)

yearly_df = pd.merge(crude_yearly, gas_yearly, on='Year')
print(yearly_df)
```

Output:

	Year	Crude_Mean	Crude_Max	Crude_Min
0	2023	77.561909	93.680000	66.739998
1	2024	75.758016	86.910004	65.750000
2	2025	64.739524	80.040001	55.270000
3	2026	61.373333	65.419998	55.990002

	Year	Gas_Mean	Gas_Max	Gas_Min
0	2023	2.606086	3.575	1.991
1	2024	2.408591	3.946	1.575
2	2025	3.619786	5.289	2.696
3	2026	3.939167	7.460	3.103

	Year	Crude_Mean	Crude_Max	Crude_Min	Gas_Mean	Gas_Max	Gas_Min
0	2023	77.561909	93.680000	66.739998	2.606086	3.575	1.991
1	2024	75.758016	86.910004	65.750000	2.408591	3.946	1.575
2	2025	64.739524	80.040001	55.270000	3.619786	5.289	2.696
3	2026	61.373333	65.419998	55.990002	3.939167	7.460	3.103

Explanation

Crude Oil

- 2023: Highest average price (~77.6 USD), with max ~93.7 and min ~66.7.
- 2024: Slight decline in average (~75.8 USD).
- 2025: Significant drop in average (~64.7 USD).

- 2026: Lowest average (~61.3 USD), showing a **downward trend**.

Natural Gas

- 2023: Average ~2.61 USD, relatively stable.
- 2024: Slight decline (~2.41 USD).
- 2025: Sharp increase (~3.62 USD), with max ~5.29.
- 2026: Further rise (~3.94 USD), with max ~7.46, **showing volatility**.

Combined View

- Crude oil shows a **steady decline** in average prices from 2023 to 2026.
- Natural gas shows **increasing volatility**, with averages rising sharply after 2024.
- This contrast highlights **different market dynamics** in energy commodities.

Moving Average:

Purpose:

To identify **short-, medium-, and long-term trends** using 20-day, 50-day, and 200-day moving averages.

Definitions:

Moving Average (MA) = Average of closing prices over a specific window of days: 20, 50, 200.

- **MA_20** → Short-term trend (20 days)
- **MA_50** → Medium-term trend (50 days)
- **MA_200** → Long-term trend (200 days)
- Smoothens daily price fluctuations and highlights overall trend.

Crude Oil:

```
# 10-Day Moving Average (first)
# Crude Oil
crude_df['MA_20'] = crude_df['Close'].rolling(window=20).mean()
crude_df['MA_50'] = crude_df['Close'].rolling(window=50).mean()
crude_df['MA_200'] = crude_df['Close'].rolling(window=200).mean()
print(crude_df[['Close', 'MA_20', 'MA_50', 'MA_200']].tail())
```

	Close	MA_20	MA_50	MA_200
1499	64.360001	62.062000	59.5428	62.32985
1501	63.959999	62.285000	59.6490	62.33455
1503	64.629997	62.458999	59.7706	62.34745
1505	62.840000	62.499999	59.8410	62.35955
1507	62.889999	62.684999	59.9260	62.38295

Explanation:

- losing prices are slightly **above the 200-day MA**, suggesting prices are trading near the long-term trend.
- MA-20 closely follows the closing price, indicating **short-term alignment**.
- MA-50 remains lower, suggesting **weaker medium-term performance**.

Natural Gas:

```
# Natural Gas
gas_df['MA_20'] = gas_df['Close'].rolling(window=20).mean()
gas_df['MA_50'] = gas_df['Close'].rolling(window=50).mean()
gas_df['MA_200'] = gas_df['Close'].rolling(window=200).mean()

print(gas_df[['Close', 'MA_20', 'MA_50', 'MA_200']].tail())
```

	Close	MA_20	MA_50	MA_200
1500	3.138	4.24245	4.23044	3.611955
1502	3.115	4.22775	4.20158	3.612845
1504	3.159	4.21475	4.16776	3.612790
1506	3.217	4.21960	4.13368	3.611945
1508	3.243	4.22535	4.10174	3.611530

Explanation:

- The closing prices (~3.1–3.2) are **below both the 20-day and 50-day averages (~4.1–4.2)**.
- This indicates that natural gas is currently trading **weaker than its short- and medium-term trends**.
- However, the **200-day average (~3.61)** is closer to the closing price, suggesting long-term stability compared to recent declines.

Matplot:

Crude Oil Moving Averages

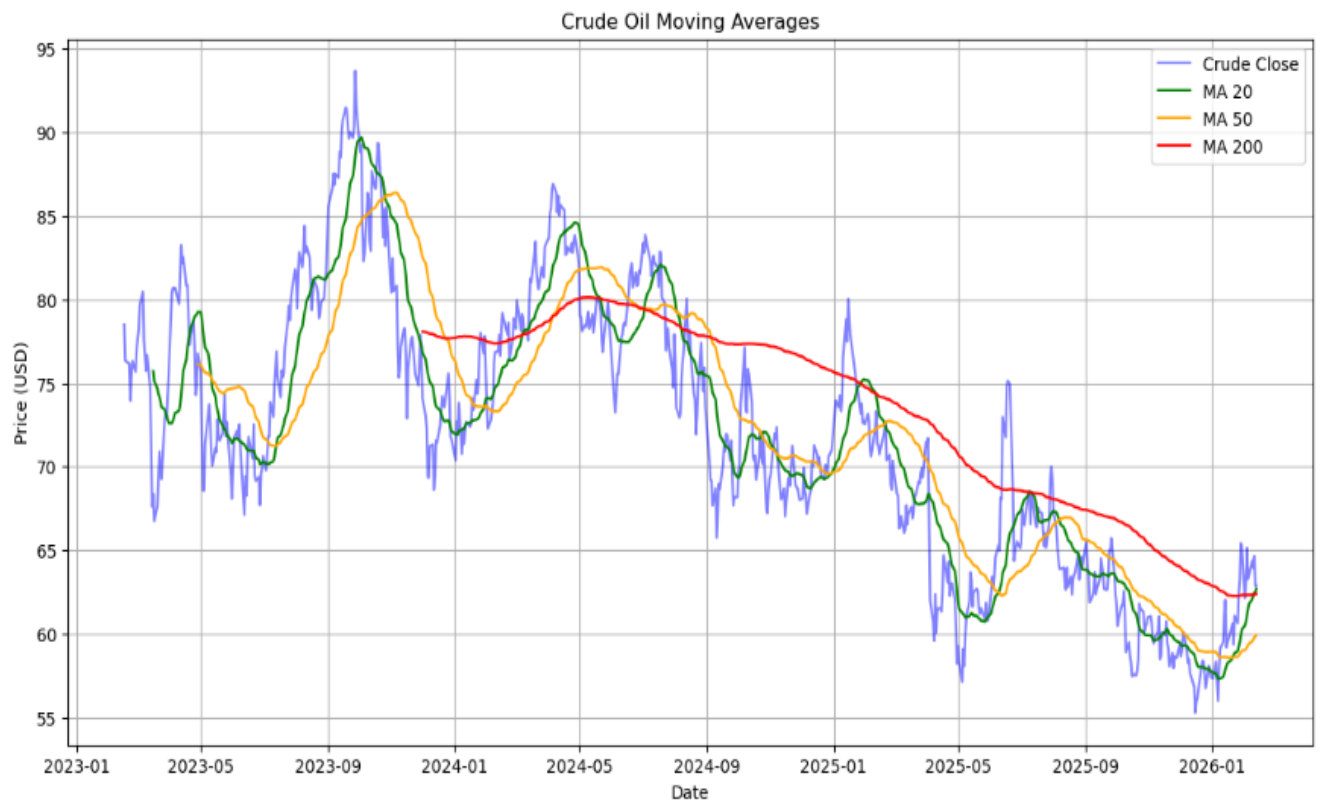
Purpose:

Shows short-, medium-, and long-term trends of crude oil prices using 20-, 50-, and 200-day moving averages. Helps visualize trend direction, smooth price fluctuations, and spot potential reversals.

```
# Crude Oil
plt.plot(crude_df['Date'], crude_df['Close'], label='Crude Close', color='blue', alpha=0.5)
plt.plot(crude_df['Date'], crude_df['MA_20'], label='MA 20', color='green')
plt.plot(crude_df['Date'], crude_df['MA_50'], label='MA 50', color='orange')
plt.plot(crude_df['Date'], crude_df['MA_200'], label='MA 200', color='red')

plt.title("Crude Oil Moving Averages")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Explanation of Components:

- **Blue Line (Crude Close):** Represents the actual daily closing price of crude oil.
- **Green Line (MA 20):** Short-term moving average, reacts quickly to price changes.
- **Orange Line (MA 50):** Medium-term moving average, smoother, shows sustained trends.
- **Red Line (MA 200):** Long-term moving average, very stable, used to confirm overall market direction.

Explanation of chart:

- The **short-term average (MA 20)** closely follows the daily price, highlighting short-term momentum.
- The **medium-term average (MA 50)** smooths fluctuations, showing broader trends.
- The **long-term average (MA 200)** acts as a baseline, confirming whether crude oil is in a bullish or bearish phase.

Natural Gas Moving Average:

Purpose:

The purpose of using moving averages is to **smooth out price fluctuations** and identify the overall trend of natural gas prices. This helps traders and analysts:

- Spot short-term vs. long-term trends.
- Detect potential turning points.
- Make better trading and investment decisions.

```
# --- Natural Gas ---
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

plt.figure(figsize=(14,6))

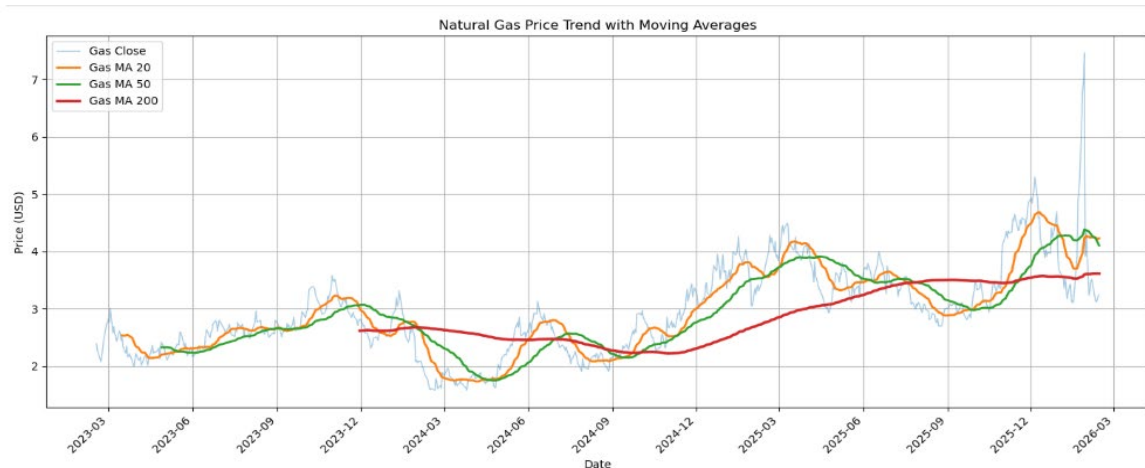
# Close price (light)
plt.plot(
    gas_df['Date'],
    gas_df['Close'],
    label='Gas Close',
    alpha=0.4,
    linewidth=1
)

# Moving Averages
plt.plot(gas_df['Date'], gas_df['MA_20'], label='Gas MA 20', linewidth=2)
plt.plot(gas_df['Date'], gas_df['MA_50'], label='Gas MA 50', linewidth=2)
plt.plot(gas_df['Date'], gas_df['MA_200'], label='Gas MA 200', linewidth=2.5)

# X-axis date formatting
ax = plt.gca()
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))

plt.xticks(rotation=45, ha='right')
plt.title("Natural Gas Price Trend with Moving Averages")
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:



Explanation of Components

- **Gas Close (Light Blue Line):** Daily closing price of natural gas.
- **Gas MA 20 (Orange Line):** 20-day moving average, showing short-term trends.
- **Gas MA 50 (Green Line):** 50-day moving average, showing medium-term trends.
- **Gas MA 200 (Red Line):** 200-day moving average, showing long-term trends

Observations from the Chart

- Between 2020 and 2023, natural gas prices showed **sharp fluctuations**.
- Short-term averages (MA 20, MA 50) reacted quickly to price changes.
- Long-term average (MA 200) provided a smoother view of the overall trend.
- Crossovers between these lines highlighted periods of **trend shifts**.

RSI (Relative Strength Index)

Means:

- **RSI** = Momentum oscillator measuring the speed and change of price movements.
- Scale: **0–100**
 - 70 → Overbought (price very high)
 - <30 → Oversold (price very low)

Purpose:

Measures momentum of crude oil and natural gas prices, highlighting overbought (>70) and oversold (<30) conditions. Helps identify potential trend reversals and support trading decisions.

```
# RSI calculation (14-day - standard)

def compute_rsi(data, period=14):
    delta = data['Close'].diff()
    gain = delta.where(delta > 0, 0)
    loss = -delta.where(delta < 0, 0)

    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()

    rs = avg_gain / avg_loss
    rsi = 100 - (100 / (1 + rs))
    return rsi

# Crude Oil RSI
crude_df['RSI_14'] = compute_rsi(crude_df, 14)

# Natural Gas RSI
gas_df['RSI_14'] = compute_rsi(gas_df, 14)
```

```
print(crude_df[['Date', 'Close', 'RSI_14']].head(20))
```

	Date	Close	RSI_14
0	2023-02-15	78.589996	NaN
2	2023-02-16	78.489998	NaN
4	2023-02-17	76.339996	NaN
6	2023-02-21	76.160004	NaN
8	2023-02-22	73.949997	NaN
10	2023-02-23	75.389999	NaN
12	2023-02-24	76.320000	NaN
14	2023-02-27	75.680000	NaN
16	2023-02-28	77.050003	NaN
18	2023-03-01	77.690002	NaN
20	2023-03-02	78.160004	NaN
22	2023-03-03	79.680000	NaN
24	2023-03-06	80.459999	NaN
26	2023-03-07	77.580002	46.701520
28	2023-03-08	76.660004	44.054242
30	2023-03-09	75.720001	41.886360
32	2023-03-10	76.680000	51.070542
34	2023-03-13	74.800003	46.131967
36	2023-03-14	71.330002	43.046721
38	2023-03-15	67.610001	31.581438

```
print(crude_df[['Date', 'Close', 'RSI_14']].tail(20))
```

	Date	Close	RSI_14
1469	2026-01-16	59.439999	59.284721
1471	2026-01-20	60.340000	58.014177
1473	2026-01-21	60.619999	59.368415
1475	2026-01-22	59.360001	56.475310
1477	2026-01-23	61.070000	61.301992
1479	2026-01-26	60.630001	57.205247
1481	2026-01-27	62.389999	65.843374
1483	2026-01-28	63.209999	72.174446
1485	2026-01-29	65.419998	72.906702
1487	2026-01-30	65.209999	69.556846
1489	2026-02-02	62.139999	57.228916
1491	2026-02-03	63.209999	55.825787
1493	2026-02-04	65.139999	58.324439
1495	2026-02-05	63.290001	61.542805
1497	2026-02-06	63.549999	61.564443
1499	2026-02-09	64.360001	61.368787
1501	2026-02-10	63.959999	59.382027
1503	2026-02-11	64.629997	65.310864
1505	2026-02-12	62.840000	55.118570
1507	2026-02-13	62.889999	56.686389

RSI Explanation:

RSI (14-day) was calculated to measure price momentum. Initial values are shown as Nan because RSI requires at least 14 days of data to compute. After that, RSI values help identify overbought (>70) and oversold (<30) conditions.

Natural Gas - RSI

```
print(gas_df[['Date', 'Close', 'RSI_14']].head(20)) print(gas_df[['Date', 'Close', 'RSI_14']].tail(20))
```

	Date	Close	RSI_14
1	2023-02-15	2.471	NaN
3	2023-02-16	2.389	NaN
5	2023-02-17	2.275	NaN
7	2023-02-21	2.073	NaN
9	2023-02-22	2.174	NaN
11	2023-02-23	2.314	NaN
13	2023-02-24	2.451	NaN
15	2023-02-27	2.731	NaN
17	2023-02-28	2.747	NaN
19	2023-03-01	2.811	NaN
21	2023-03-02	2.765	NaN
23	2023-03-03	3.009	NaN
25	2023-03-06	2.572	NaN
27	2023-03-07	2.687	55.460062
29	2023-03-08	2.551	51.892151
31	2023-03-09	2.543	53.774511
33	2023-03-10	2.430	53.800882
35	2023-03-13	2.606	63.238947
37	2023-03-14	2.573	60.257068
39	2023-03-15	2.439	53.223311

	Date	Close	RSI_14
1470	2026-01-16	3.103	27.210390
1472	2026-01-20	3.907	38.014758
1474	2026-01-21	4.875	62.874251
1476	2026-01-22	5.045	70.038330
1478	2026-01-23	5.275	73.318317
1480	2026-01-26	6.800	82.881793
1482	2026-01-27	6.954	86.301366
1484	2026-01-28	7.460	87.157692
1486	2026-01-29	3.918	52.930381
1488	2026-01-30	4.354	56.644612
1490	2026-02-02	3.237	49.121912
1492	2026-02-03	3.311	49.452223
1494	2026-02-04	3.465	51.775970
1496	2026-02-05	3.509	51.954047
1498	2026-02-06	3.422	51.625726
1500	2026-02-09	3.138	45.861586
1502	2026-02-10	3.115	39.456027
1504	2026-02-11	3.159	38.527980
1506	2026-02-12	3.217	37.214215
1508	2026-02-13	3.243	22.843182

Explanation:

RSI (14-day) was calculated to analyse momentum in natural gas prices. Initial values appear as Nan because RSI requires a minimum of 14 days of data. RSI values above 70 indicate overbought conditions, while values below 30 indicate oversold conditions. In 2026, natural gas showed sharp RSI fluctuations, reflecting high volatility.

- Example: 2026-01-16 → Close = 3.103, RSI_14 = 27.21 → Oversold
- Example: 2026-01-28 → Close = 7.460, RSI_14 = 87.16 → Overbought

Matplot:

```
plt.figure(figsize=(14,6))

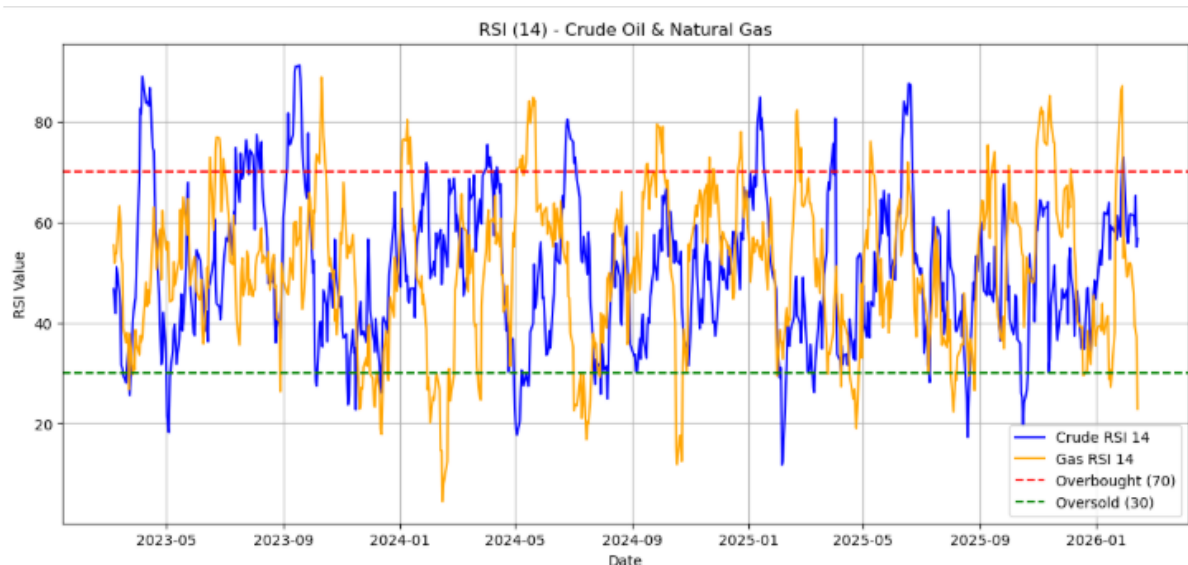
# Crude Oil RSI
plt.plot(crude_df['Date'], crude_df['RSI_14'], label='Crude RSI 14', color='blue')

# Natural Gas RSI
plt.plot(gas_df['Date'], gas_df['RSI_14'], label='Gas RSI 14', color='orange')

# Add Overbought/Oversold Lines
plt.axhline(70, color='red', linestyle='--', label='Overbought (70)')
plt.axhline(30, color='green', linestyle='--', label='Oversold (30)')

plt.title("RSI (14) - Crude Oil & Natural Gas")
plt.xlabel("Date")
plt.ylabel("RSI Value")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



Observations from the Chart

- Both **Crude Oil (Blue Line)** and **Natural Gas (Orange Line)** fluctuate between overbought and oversold zones from 2023 to 2026.

- Periods where RSI rises above 70 suggest strong bullish momentum but also risk of correction.
- Periods where RSI falls below 30 suggest bearish momentum but also potential for recovery.
- The chart highlights how momentum shifts differ between crude oil and natural gas, reflecting their unique market drivers (supply, demand, geopolitical factors, weather).

MACD:

Purpose:

MACD shows trend direction and momentum for Crude Oil and Natural Gas. Bullish, Bearish, or Neutral signals help identify trend changes and support trading or forecasting decisions

```
|# Two EMA difference

# Useful for Trend change identify

def add_technical_macd(data):
    # Sort by Date (VERY IMPORTANT)
    data = data.sort_values('Date')

    # Exponential Moving Averages
    data['EMA12'] = data['Close'].ewm(span=12, adjust=False).mean()
    data['EMA26'] = data['Close'].ewm(span=26, adjust=False).mean()

    # MACD & Signal
    data['MACD'] = data['EMA12'] - data['EMA26']
    data['Signal_Line'] = data['MACD'].ewm(span=9, adjust=False).mean()
    data['MACD_Hist'] = data['MACD'] - data['Signal_Line']

    # MACD Signal Label
    data['MACD_Signal'] = 'Neutral'
    data.loc[data['MACD'] > data['Signal_Line'], 'MACD_Signal'] = 'Bullish'
    data.loc[data['MACD'] < data['Signal_Line'], 'MACD_Signal'] = 'Bearish'

    # Quick check
    print(
        data[['Date', 'Close', 'MACD', 'Signal_Line', 'MACD_Hist', 'MACD_Signal']]
        .tail(10)
    )

    return data

crude_df = add_technical_macd(crude_df)
gas_df   = add_technical_macd(gas_df)
```

Crude Oil:

Output:

	Date	Close	MACD	Signal_Line	MACD_Hist	MACD_Signal
1489	2026-02-02	62.139999	1.397331	0.999742	0.397589	Bullish
1491	2026-02-03	63.209999	1.388001	1.077394	0.310608	Bullish
1493	2026-02-04	65.139999	1.518834	1.165682	0.353153	Bullish
1495	2026-02-05	63.290001	1.456452	1.223836	0.232616	Bullish
1497	2026-02-06	63.549999	1.411720	1.261413	0.150307	Bullish
1499	2026-02-09	64.360001	1.425201	1.294170	0.131030	Bullish
1501	2026-02-10	63.959999	1.387612	1.312859	0.074754	Bullish
1503	2026-02-11	64.629997	1.395796	1.329446	0.066350	Bullish
1505	2026-02-12	62.840000	1.243510	1.312259	-0.068749	Bearish
1507	2026-02-13	62.889999	1.114015	1.272610	-0.158595	Bearish

Columns

- **Date:** Trading day
- **Close:** Closing price of the commodity on that day
- **MACD:** Difference between 12-day EMA and 26-day EMA of closing price
- **Signal Line:** 9-day EMA of MACD
- **MACD Hist:** MACD – Signal Line; shows momentum strength
- **MACD Signal:**
 - Bullish: $\text{MACD} > \text{Signal Line}$ → potential upward trend
 - Bearish: $\text{MACD} < \text{Signal Line}$ → potential downward trend
 - Neutral: $\text{MACD} \approx \text{Signal Line}$ → trend unclear

Explanation:

- Bullish signals (Crude Oil, e.g., 2026-02-02 to 2026-02-11): Prices likely trending upward; MACD above Signal Line; histogram positive.
- Bearish signals (Crude Oil, 2026-02-12 onwards & Natural Gas 2026-02-02 onwards): Prices likely trending downward; MACD below Signal Line; histogram negative.
- Histogram magnitude: Shows momentum strength; higher positive → strong uptrend, higher negative → strong downtrend.
- Trend Reversal Identification: When MACD crosses Signal Line from below → potential bullish reversal; from above → bearish reversal.

Natural Gas:

	Date	Close	MACD	Signal_Line	MACD_Hist	MACD_Signal
1490	2026-02-02	3.237	0.233623	0.272041	-0.038417	Bearish
1492	2026-02-03	3.311	0.105058	0.238644	-0.133586	Bearish
1494	2026-02-04	3.465	0.015418	0.193999	-0.178581	Bearish
1496	2026-02-05	3.509	-0.051479	0.144903	-0.196382	Bearish
1498	2026-02-06	3.422	-0.110244	0.093874	-0.204118	Bearish
1500	2026-02-09	3.138	-0.177684	0.039562	-0.217247	Bearish
1502	2026-02-10	3.115	-0.230332	-0.014417	-0.215915	Bearish
1504	2026-02-11	3.159	-0.265445	-0.064622	-0.200823	Bearish
1506	2026-02-12	3.217	-0.285304	-0.108759	-0.176545	Bearish
1508	2026-02-13	3.243	-0.295537	-0.146114	-0.149423	Bearish

Explanation:

- 2026-02-02 to 2026-02-13: All entries show Bearish
- $MACD < \text{Signal Line}$
- Histogram negative
- Prices trend downward or weak momentum (3.237 → 3.243, small fluctuations)
- Histogram magnitude: Indicates momentum strength
- More negative → stronger downward trend
- Less negative → weak downward momentum
- Trend Reversal: If MACD crosses above Signal Line → bullish trend possible in future

Matplot:

```
import matplotlib.pyplot as plt

def plot_macd(data, title):
    plt.figure(figsize=(14,6))

    # MACD & Signal
    plt.plot(data["Date"], data["MACD"], label="MACD", linewidth=2)
    plt.plot(data["Date"], data["Signal_Line"], label="Signal Line", linewidth=2)

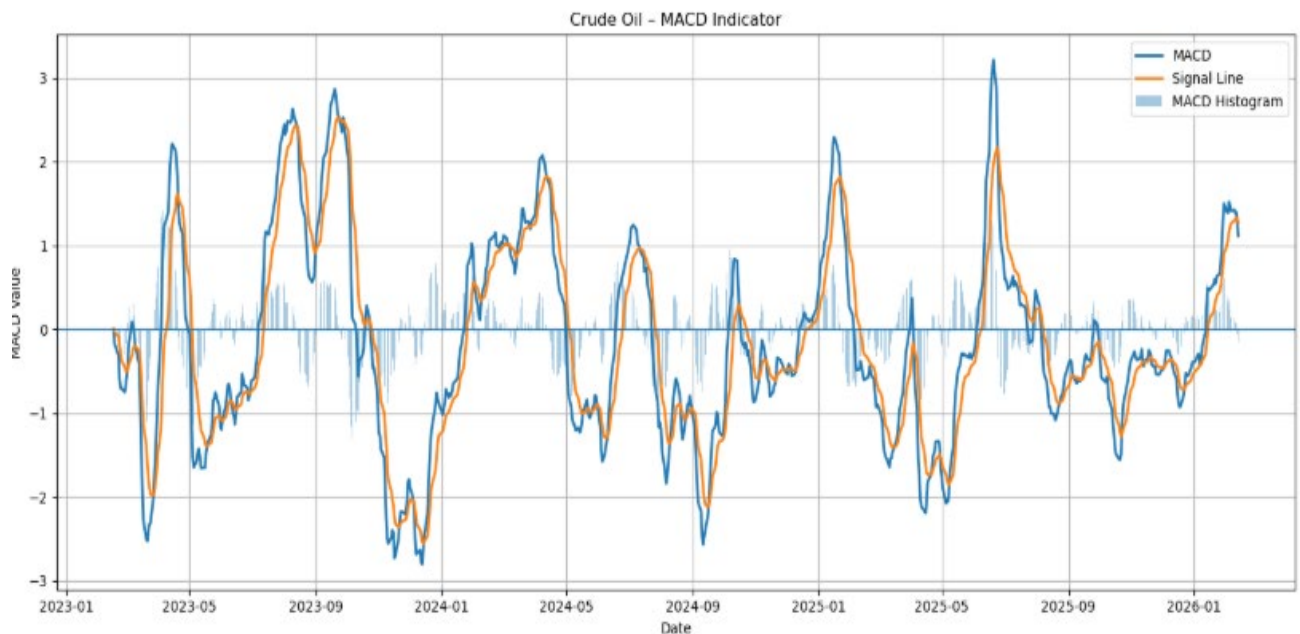
    # Histogram
    plt.bar(
        data["Date"],
        data["MACD_Hist"],
        label="MACD Histogram",
        alpha=0.4
    )

    # Zero Line
    plt.axhline(0, linewidth=1)

    plt.title(title)
    plt.xlabel("Date")
    plt.ylabel("MACD Value")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

crude_df = add_technical_macd(crude_df)
plot_macd(crude_df, "Crude Oil - MACD Indicator")
```

Output:



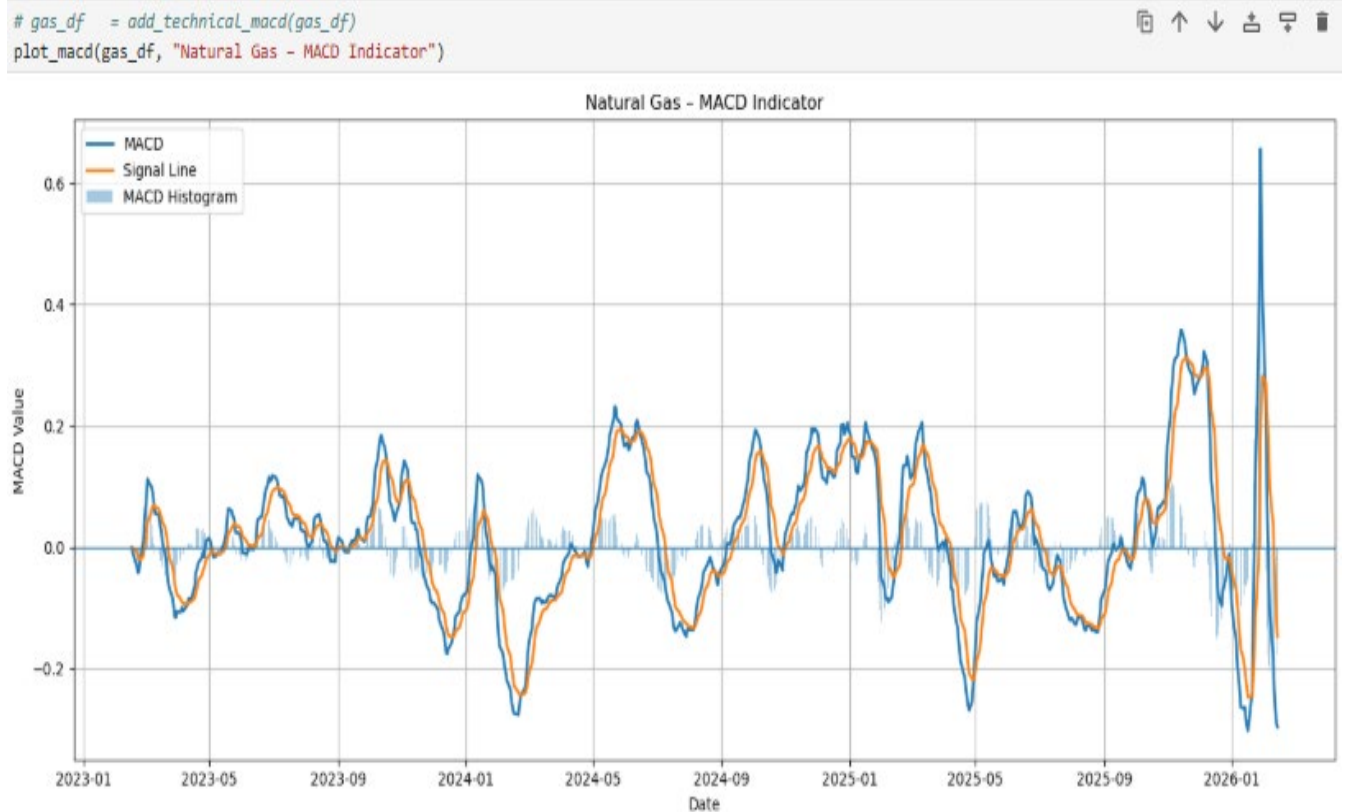
Explanation of Components

- **MACD Line (Blue):** Calculated as the difference between the 12-day and 26-day exponential moving averages (EMA). Shows short-term momentum.
- **Signal Line (Orange):** A 9-day EMA of the MACD line. Acts as a trigger for buy/sell signals.
- **MACD Histogram (Light Blue Bars):** Represents the difference between the MACD line and the Signal line. Positive bars indicate bullish momentum; negative bars indicate bearish momentum.

Observations from the Chart

- Between 2023 and 2026, multiple crossovers occur, marking shifts in crude oil momentum.
- Periods where the MACD line rises above the Signal line align with bullish phases in oil prices.
- Periods where the MACD line falls below the Signal line align with bearish corrections.
- The histogram provides a quick visual of momentum strength, with spikes showing strong moves.

MACD -Natural gas



Explanation of Components

- **MACD Line (Blue):** Difference between the 12-day EMA and 26-day EMA. Shows short-term momentum.
- **Signal Line (Orange):** A 9-day EMA of the MACD line. Acts as a trigger for buy/sell signals.
- **Histogram (Light Blue Bars):** Difference between MACD and Signal line. Positive bars = bullish momentum; negative bars = bearish momentum

Observations from the Chart

- Between 2023 and 2026, multiple crossovers occur, marking shifts in natural gas momentum.
- Bullish phases appear when the MACD line rises above the Signal line.
- Bearish phases appear when the MACD line falls below the Signal line.
- The histogram highlights the strength of these moves, with spikes showing strong momentum shifts.

Yearly Trend

Purpose

1. **Yearly Trend Summary:** Shows how many days each year were trending up, down, or sideways.
2. **Trend Analysis:** Helps identify years dominated by bullish or bearish trends.
3. **Visual & Report Support:** Useful for dashboards, charts, and annual market behaviour analysis.
4. **Decision Support:** Traders or analysts can see which years had sustained trends, helping in strategy planning.

- **Year:** Trading year
- **Downtrend (%):** % of days price was below MA_20 → bearish trend
- **Uptrend (%):** % of days price was above MA_20 → bullish trend
- **Sideways (%):** % of days price was near MA_20 → neutral/consolidation

```
# Moving Average
crude_df['MA_20'] = crude_df['Close'].rolling(20).mean()

# Trend
crude_df['Trend'] = 'Sideways'
crude_df.loc[crude_df['Close'] > crude_df['MA_20'], 'Trend'] = 'Uptrend'
crude_df.loc[crude_df['Close'] < crude_df['MA_20'], 'Trend'] = 'Downtrend'

crude_df['Year'] = pd.to_datetime(crude_df['Date']).dt.year

crude_yearly = (
    crude_df.groupby(['Year', 'Trend'])
    .size()
    .reset_index(name='Days')
)

crude_yearly['Percentage'] = (
    crude_yearly['Days'] /
    crude_yearly.groupby('Year')['Days'].transform('sum')
) * 100

crude_trend_table = crude_yearly.pivot(
    index='Year',
    columns='Trend',
    values='Percentage'
).round(2)

crude_trend_table
```

Output:

Trend	Downtrend	Sideways	Uptrend
Year			
2023	51.60	8.68	39.73
2024	50.00	NaN	50.00
2025	64.29	NaN	35.71
2026	10.00	NaN	90.00

- **2023:** 51.60% Downtrend, 39.73% Uptrend, 8.68% Sideways → Slightly bearish year with some bullish periods
- **2024:** 50% Uptrend, 50% Downtrend, 0% Sideways → Balanced trend year
- **2025:** 64.29% Downtrend, 35.71% Uptrend → Mostly bearish year
- **2026:** 90% Uptrend, 10% Downtrend → Strong bullish trend so far

Natural Gas trend:

```
gas_df['MA_20'] = gas_df['Close'].rolling(20).mean()

gas_df['Trend'] = 'Sideways'
gas_df.loc[gas_df['Close'] > gas_df['MA_20'], 'Trend'] = 'Uptrend'
gas_df.loc[gas_df['Close'] < gas_df['MA_20'], 'Trend'] = 'Downtrend'
gas_df['Year'] = pd.to_datetime(gas_df['Date']).dt.year
gas_yearly = (
    gas_df.groupby(['Year', 'Trend'])
    .size()
    .reset_index(name='Days')
)

gas_yearly['Percentage'] = (
    gas_yearly['Days'] /
    gas_yearly.groupby('Year')['Days'].transform('sum')
) * 100

gas_trend_table = gas_yearly.pivot(
    index='Year',
    columns='Trend',
    values='Percentage'
).round(2)

gas_trend_table
```

Trend	Downtrend	Sideways	Uptrend
Year			
2023	48.18	8.64	43.18
2024	38.89	NaN	61.11
2025	50.79	NaN	49.21
2026	73.33	NaN	26.67

Explanation:

- **2023:** 48.18% Downtrend, 43.18% Uptrend, 8.64% Sideways → Slightly bearish year
- **2024:** 38.89% Downtrend, 61.11% Uptrend → Mostly bullish year
- **2025:** 50.79% Downtrend, 49.21% Uptrend → Almost balanced year
- **2026:** 73.33% Downtrend, 26.67% Uptrend → Strong bearish trend so far

Volatility Pattern (High – Low Range):

- **High Volatility:** Day range $>$ average range $\rightarrow$ price fluctuated more than usual
- **Low Volatility:** Day range $\leq$ average range $\rightarrow$ price relatively stable

Purpose

- **Identify Price Fluctuations:** Highlights which days had unusually large or small price movements.
- **Support Risk Analysis:** High volatility days indicate higher risk, Low volatility indicates stability.
- **Trend & Market Behaviour:** Helps understand market momentum and periods of high uncertainty.
- **Dashboard & Reporting:** Useful for plotting volatility patterns and analysing commodity behaviour visually.

```
# VOLATILITY PATTERN (High-Low range)

def volatility_pattern(data):
    # Sort by date (safety)
    data = data.sort_values("Date").copy()

    # Day range
    data["Day_Range"] = data["High"] - data["Low"]

    # Average range
    avg_range = data["Day_Range"].mean()

    # Label volatility
    data["Volatility_Pattern"] = "Low Volatility"
    data.loc[
        data["Day_Range"] > avg_range,
        "Volatility_Pattern"
    ] = "High Volatility"

    # Show output
    print("Average Day Range:", round(avg_range, 2))
    display(data[["Date", "Day_Range", "Volatility_Pattern"]].tail(10))

    return data
```

Crude Oil Output:

```
crude_df = volatility_pattern(crude_df)
```

Average Day Range: 2.14

	Date	Day_Range	Volatility_Pattern
1489	2026-02-02	3.349998	High Volatility
1491	2026-02-03	3.090000	High Volatility
1493	2026-02-04	2.669998	High Volatility
1495	2026-02-05	2.019997	Low Volatility
1497	2026-02-06	2.380001	High Volatility
1499	2026-02-09	2.259998	High Volatility
1501	2026-02-10	1.059998	Low Volatility
1503	2026-02-11	1.680000	Low Volatility
1505	2026-02-12	2.709999	High Volatility
1507	2026-02-13	1.119999	Low Volatility

Explanation:

- Crude oil experienced **frequent high volatility days** (Feb 2–4, Feb 6, Feb 12).
- **Low volatility days** (Feb 5, Feb 10–11, Feb 13) show smaller price movements.
- This mix indicates alternating periods of stability and sharp fluctuations.

Business Relevance

- **Traders** use volatility patterns to adjust risk exposure.
- **High volatility days** → higher risk but also higher profit opportunities.
- **Low volatility days** → safer but limited trading opportunities.

Natural gas

```
gas_df = volatility_pattern(gas_df)
```

Average Day Range: 0.18

	Date	Day_Range	Volatility_Pattern
1490	2026-02-02	0.581	High Volatility
1492	2026-02-03	0.228	High Volatility
1494	2026-02-04	0.284	High Volatility
1496	2026-02-05	0.245	High Volatility
1498	2026-02-06	0.272	High Volatility
1500	2026-02-09	0.154	Low Volatility
1502	2026-02-10	0.153	Low Volatility
1504	2026-02-11	0.194	High Volatility
1506	2026-02-12	0.136	Low Volatility
1508	2026-02-13	0.164	Low Volatility

Explanation

- **2026-02-02 to 2026-02-06:** High Volatility (0.228 – 0.581) → big price swings, risky or active market
- **2026-02-09 to 2026-02-13:** Mostly Low Volatility (0.136 – 0.164) → stable prices, low fluctuations
- **2026-02-11:** High Volatility (0.194) → one-off spike above average
- **High Volatility Days:** Market was highly active; traders may experience larger profits or losses.
- **Low Volatility Days:** Market stable; fewer opportunities for quick trading.

Golden Cross & Death Cross:

Purpose

1. **Identify Trend Reversals:** Helps detect major trend shifts in the market.
2. **Long-term Market Analysis:** Useful for assessing bullish or bearish cycles over months or years.

3. **Trading Signals:** Traders use Golden Cross for entry (buy) and Death Cross for exit (sell).

4. **Visualization & Reporting:** Supports plotting trend shifts and highlighting key market signals.

1. Calculates two moving averages:

- **MA_50:** 50-day moving average
- **MA_200:** 200-day moving average

2. Detects **crossover signals**:

- **Golden Cross:** MA_50 crosses above MA_200 → potential long-term bullish trend
- **Death Cross:** MA_50 crosses below MA_200 → potential long-term bearish trend

3. Labels each crossover day in **Cross Signal** column

```
df["Signal"] = 0
```

```
def golden_death_cross(data):
    data = data.sort_values("Date").copy()

    # Moving averages
    data["MA_50"] = data["Close"].rolling(50).mean()
    data["MA_200"] = data["Close"].rolling(200).mean()

    # Default
    data["Cross_Signal"] = "None"

    # Golden Cross
    data.loc[
        (data["MA_50"] > data["MA_200"]) &
        (data["MA_50"].shift(1) <= data["MA_200"].shift(1)),
        "Cross_Signal"
    ] = "Golden Cross"

    # Death Cross
    data.loc[
        (data["MA_50"] < data["MA_200"]) &
        (data["MA_50"].shift(1) >= data["MA_200"].shift(1)),
        "Cross_Signal"
    ] = "Death Cross"

    # Output
    display(
        data.loc[
            data["Cross_Signal"] != "None",
            ["Date", "Close", "MA_50", "MA_200", "Cross_Signal"]
        ].tail(10)
    )

    return data
```

Output:

Crude – Golden Death Cross

```
crude_df = golden_death_cross(crude_df)
```

	Date	Close	MA_50	MA_200	Cross_Signal
433	2023-12-26	75.570000	77.5262	77.67745	Death Cross
571	2024-04-05	86.910004	79.0782	79.02415	Golden Cross
683	2024-06-26	80.900002	79.5316	79.55905	Death Cross
691	2024-07-02	82.809998	79.4426	79.43235	Golden Cross
781	2024-09-05	69.150002	77.6600	77.71635	Death Cross

Explanation:

- On **2023-12-26**, a **Death Cross** indicated weakness in crude oil prices.
- On **2024-04-05**, a **Golden Cross** suggested a bullish recovery.
- The alternating signals in 2024 (April, June, July, September) show **volatile market conditions** with frequent trend reversals.

Business Relevance

- **Traders** use Golden/Death Cross signals to time entry and exit points.
- **Investors** view Golden Crosses as long-term bullish confirmation, while Death Crosses warn of extended declines.
- Frequent crossovers in 2024 highlight crude oil's **instability**, making risk management essential.

Natural Gas – Golden Death Cross:

```
gas_df = golden_death_cross(gas_df)
```

	Date	Close	MA_50	MA_200	Cross_Signal
482	2024-01-31	2.100	2.66624	2.673700	Death Cross
692	2024-07-02	2.435	2.48040	2.467085	Golden Cross
768	2024-08-26	1.956	2.27764	2.290595	Death Cross
820	2024-10-02	2.886	2.24168	2.238265	Golden Cross
1232	2025-07-29	3.081	3.45658	3.457620	Death Cross

Explanation

- On **2024-01-31**, a **Death Cross** indicated weakness in natural gas prices.

- On **2024-07-02**, a **Golden Cross** suggested recovery.
- On **2024-08-26**, another **Death Cross** showed renewed weakness.
- On **2024-10-22**, a **Golden Cross** signalled bullish momentum.
- By **2025-07-29**, a **Death Cross** again marked a bearish decline.

Matplot:

Purpose

1. **Visual Trend Analysis:** Quickly shows where bullish (Golden Cross) and bearish (Death Cross) trends start.
2. **Market Timing Support:** Traders can see potential entry (Golden) and exit (Death) points.
3. **Comparison with Price:** Helps understand how price reacts after crossovers.
4. **Report & Dashboard:** Makes it easy to include in presentations or Word documents for clear trend illustration.

```
import matplotlib.pyplot as plt
def plot_golden_death_cross(data, title):
    data = data.sort_values("Date")

    plt.figure(figsize=(14,6))

    # Close price
    plt.plot(data["Date"], data["Close"],
             label="Close Price", color="black", alpha=0.6)

    # Moving averages
    plt.plot(data["Date"], data["MA_50"],
             label="MA 50", color="blue", linewidth=2)
    plt.plot(data["Date"], data["MA_200"],
             label="MA 200", color="red", linewidth=2)

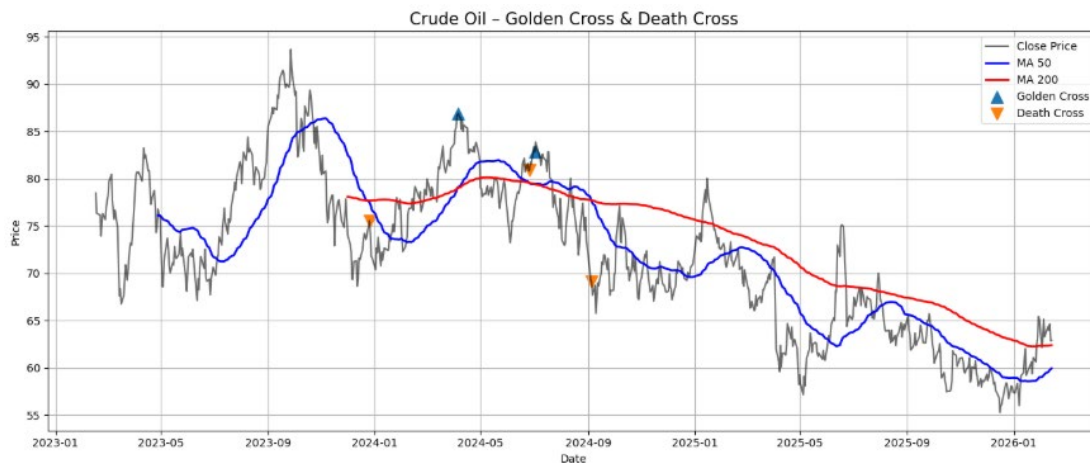
    # Golden Cross points
    golden = data[data["Cross_Signal"] == "Golden Cross"]
    plt.scatter(golden["Date"], golden["Close"],
               marker="^", s=120, label="Golden Cross")

    # Death Cross points
    death = data[data["Cross_Signal"] == "Death Cross"]
    plt.scatter(death["Date"], death["Close"],
               marker="v", s=120, label="Death Cross")

    plt.title(title, fontsize=15)
    plt.xlabel("Date")
    plt.ylabel("Price")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```

```
plot_golden_death_cross(  
    crude_df,  
    "Crude Oil - Golden Cross & Death Cross"  
)
```

Output:



Lines on the Chart:

- **Black Line (Close Price):** This shows the actual daily closing price of crude oil over time.
- **Blue Line (MA 50):** This is the 50-day moving average — it smooths out short-term price fluctuations and shows the short-term trend.
- **Red Line (MA 200):** This is the 200-day moving average — it reflects the long-term trend of the market.

Golden Cross (Blue Up Arrow):

- A bullish signal.
- Happens when the 50-day MA crosses above the 200-day MA.
- Indicates a potential upward trend or rally in prices.
- In the chart, this occurred around early 2024, followed by a price increase — confirming the bullish momentum.

Death Cross (Orange Down Arrow):

- A bearish signal.
- Happens when the 50-day MA crosses below the 200-day MA.
- Suggests a possible downward trend or market weakness.

- This appears around mid-to-late 2025 in the chart, followed by a decline in crude oil prices.

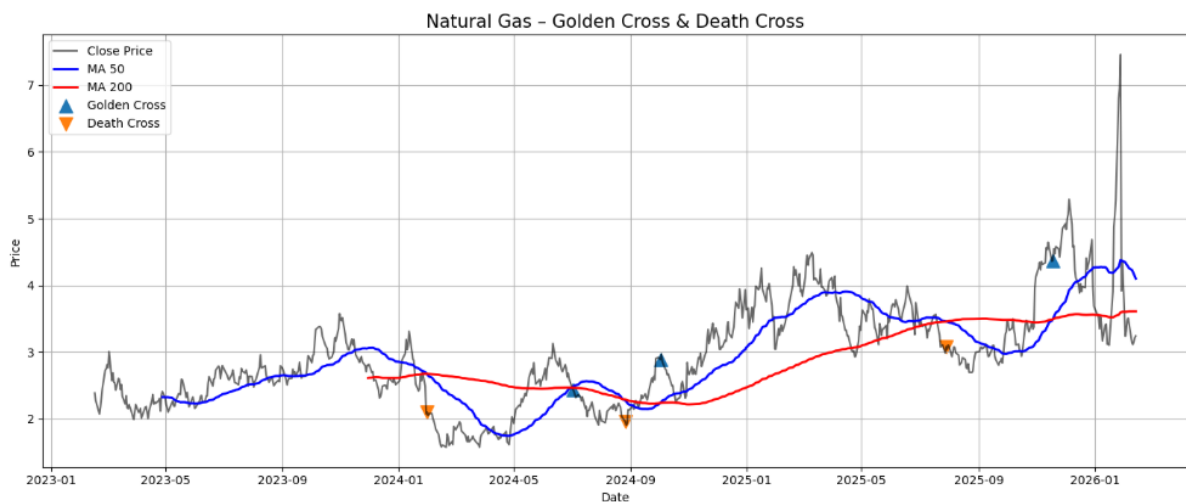
Business Impact

- **Golden Cross:** “Crude oil entered a bullish phase, indicating potential opportunities for energy sector growth.
- **Death Cross:** “Market sentiment turned bearish, signalling risks for investors and downstream industries.

Natural Gas

```
plot_golden_death_cross(  
    gas_df,  
    "Natural Gas - Golden Cross & Death Cross"  
)
```

Output:



Explanation of Indicators

- **50-Day Moving Average (MA 50 – Blue Line):** Short-term trend indicator.
- **200-Day Moving Average (MA 200 – Red Line):** Long-term trend indicator.
- **Close Price (Black Line):** Actual daily market price of natural gas.

Golden Cross (Blue Upward Triangle)

- Occurs when MA 50 crosses **above** MA 200.

- Suggests **bullish momentum** and potential price increases.
- Often interpreted as a **buy signal**.

Death Cross (Orange Downward Triangle)

- Occurs when MA 50 crosses **below** MA 200.
- Suggests **bearish momentum** and potential price declines.
- Often interpreted as a **sell signal** or cautionary sign.

Observations from the Chart

- Several Golden Crosses and Death Crosses appear between 2023 and 2026.
- Each crossover marks a possible **trend reversal** in natural gas prices.
- A sharp spike in late 2025 shows how external factors (supply, demand, geopolitical events) can cause sudden price movements, even beyond technical signals.
- Golden Crosses align with upward phases, while Death Crosses align with downward corrections.

MYSQL

INSTALL MYSQL CONNECTOR

```
pip install mysql-connector-python sqlalchemy pymysql
```

```
Requirement already satisfied: mysql-connector-python in c:\users\hp\anaconda3\lib\site-packages (9.5.0)
Requirement already satisfied: sqlalchemy in c:\users\hp\anaconda3\lib\site-packages (2.0.43)
Requirement already satisfied: pymysql in c:\users\hp\anaconda3\lib\site-packages (1.1.2)
Requirement already satisfied: greenlet>=1 in c:\users\hp\anaconda3\lib\site-packages (from sqlalchemy) (3.2.4)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\hp\anaconda3\lib\site-packages (from sqlalchemy) (4.15.0)
Note: you may need to restart the kernel to use updated packages.
```

Connect to MySQL

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="123456",
    database="commodity_db"
)

print("Connected to MySQL successfully")
```

Connected to MySQL successfully

LOAD DATA:

```
# DataFrame → MySQL table save

df.to_sql(
    name="Crude_Natural_Gas", # table name only
    con=engine,
    if_exists="replace",
    index=False
)
print("Data loaded successfully into MySQL!")
```

Data loaded successfully into MySQL!

Create Engine:

```
from sqlalchemy import create_engine
import pandas as pd
file_path = r"C:\Users\HP\Crude_NaturalGas_3Years.csv"
engine = create_engine(
    "mysql+pymysql://root:123456@localhost/commodity_db"
)

print("Engine created")
```

Engine created

Check first 5 Rows from MySQL Table

```
# Check first 5 rows from MySQL table
from sqlalchemy import text

# Check first 5 rows from MySQL table
with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM Crude_Natural_Gas LIMIT 5;"))
    for row in result:
        print(row)
```

```
('2023-02-15', 'Crude Oil', 78.83000183105469, 79.1500015258789, 77.25, 78.58999633789062, 264486, -0.3044595808565787)
('2023-02-15', 'Natural Gas', 2.6029999256134038, 2.6070001125335693, 2.4460000099182129, 2.470999956130981, 147505, -5.071070812701454)
('2023-02-16', 'Crude Oil', 78.54000091552734, 79.54000091552734, 77.91999816894531, 78.48999786376953, 117600, -0.0636657132352119)
('2023-02-16', 'Natural Gas', 2.46399998664856, 2.559000015258789, 2.3580000400543213, 2.388999938964844, 141459, -3.043833120540235)
('2023-02-17', 'Crude Oil', 78.06999969482422, 78.25, 75.05999755859375, 76.33999633789062, 93267, -2.2159643444295893)
```

SQL Query:

```
1 • SELECT * FROM commodity_db.crude_natural_gas;
2
```

	Date	Commodity	Open	High	Low	Close	Volume	Change%
▶	2023-02-15	Crude Oil	78.83000183105469	79.1500015258789	77.25	78.58999633789062	264486	-0.3044595808565787
	2023-02-16	Crude Oil	78.54000091552734	79.54000091552734	77.91999816894531	78.48999786376953	117600	-0.0636657132352119
	2023-02-17	Crude Oil	78.06999969482422	78.25	75.05999755859375	76.33999633789062	93267	-2.2159643444295893
	2023-02-21	Crude Oil	76.52999877929688	77.51000213623047	75.69000244140625	76.16000366210938	402351	-0.4834641618831335
	2023-02-22	Crude Oil	76.11000061035156	76.55000305175781	73.80000305175781	73.94999694824219	332018	-2.8380024238438875

Crude Oil & Natural Gas

Daily High-Low range:

Purpose

1. **Track Daily Price Movement:** Understand how each commodity moved day-to-day.
2. **Analyse Volatility:** Daily High-Low range shows intra-day swings.
3. **Performance Metrics:** Daily % change helps compare relative price movement.
4. **Historical Analysis:** Last 3 years' data is useful for trends, seasonality, and reporting

```
43 -- Daily % change
44 • SELECT Date,Commodity,Open,High,Low,Close,Volume,
45 ROUND((((Close - Open) / Open) * 100, 2) AS Daily_Change_Percent,
46 ROUND((High - Low), 2) AS Daily_High_Low_Range
47 FROM commodity_db.crude_natural_gas WHERE Commodity IN ('Crude Oil', 'Natural Gas')
48 AND Date >= DATE_SUB(CURDATE(), INTERVAL 3 YEAR)
49 ORDER BY Date ASC, Commodity;
```

Output:

Result Grid		Filter Rows:		Export:	Wrap Cell Contents:		Fetch rows:	
	Date	Commodity	Open	High	Low	Close	Volume	Daily_Change_Percent
▶	2023-02-23	Crude Oil	73.91999816894531	75.98999786376953	73.83000183105469	75.38999938964844	333838	1.99
	2023-02-23	Natural Gas	2.1610000133514404	2.377000093460083	2.0950000286102295	2.313999891281128	57085	7.08
	2023-02-24	Crude Oil	75.5999984741211	76.62999725341797	74.08999633789062	76.31999969482422	329254	0.95
	2023-02-24	Natural Gas	2.450000286102295	2.5769999027252197	2.306999921798706	2.4509999752044678	168130	4.52

Explanation

This query retrieves daily Open, High, Low, Close, and Volume for Crude Oil and Natural Gas over the last 3 years, along with daily % change and intra-day High-Low range for volatility and performance analysis.

Highest Close:

Purpose

1. **Identify Peak Prices:** Shows the maximum closing price for each commodity.
2. **Historical Analysis:** Helps compare peak values of Crude Oil vs Natural Gas.

3. **Trading Insight:** Traders and analysts can benchmark highest historical prices for strategy and reporting.

```
52  -- Highest Close
53  •  SELECT Commodity, Date, Close FROM commodity_db.crude_natural_gas AS t1
54  WHERE Close = (SELECT MAX(Close) FROM commodity_db.crude_natural_gas
55  WHERE Commodity = t1.Commodity);
```

Result Grid			
Filter Rows:			
	Commodity	Date	Close
▶	Crude Oil	2023-09-27	93.68000030517578
	Natural Gas	2026-01-28	7.460000038146973

Explanation:

This table summarizes the highest closing price for Crude Oil and Natural Gas, helping to identify historical peak values for market analysis and trading insights.

Lowest Close

Purpose

1. **Identify Minimum Prices:** Shows the minimum closing price for each commodity.
2. **Historical Analysis:** Helps compare lowest values of Crude Oil vs Natural Gas.
3. **Trading Insight:** Useful for benchmarking historical lows for strategy, risk management, and reporting.

```
SELECT Commodity, Date, Close FROM commodity_db.crude_natural_gas AS t1
WHERE Close = (SELECT MIN(Close) FROM commodity_db.crude_natural_gas
WHERE Commodity = t1.Commodity);
```

Output:

Result Grid			
Filter Rows:			
	Commodity	Date	Close
▶	Natural Gas	2024-03-26	1.5750000476837158
	Crude Oil	2025-12-16	55.27000045776367

Explanation

This query retrieves the lowest closing price and its date for each commodity, supporting historical low analysis and market comparison.

Yearly Summary:

Purpose

- 1. **Yearly Performance:** Understand average price levels and trends year by year.
- 2. **Volatility Analysis:** Shows which years had higher price fluctuations.
- 3. **Historical Peaks & Lows:** Max/Min Close helps identify yearly extremes.
- 4. **Decision Support:** Useful for analysts and traders for trend evaluation, reporting, and strategy planning.

86 -- GROUP BY for yearly summary

87 • SELECT Commodity, YEAR(Date) AS Year,

88 ROUND(AVG(Close), 2) AS Avg_Close, ROUND(STDDEV(Close), 2) AS Volatility,

89 MAX(Close) AS Max_Close,

90 MIN(Close) AS Min_Close

91 FROM commodity_db.crude_natural_gas WHERE Commodity IN ('Crude Oil', 'Natural Gas')

92 GROUP BY Commodity, YEAR(Date) ORDER BY Commodity, Year;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	Commodity	Year	Avg_Close	Volatility	Max_Close	Min_Close
▶	Crude Oil	2023	77.56	6.32	93.68000030517578	66.73999786376953
	Crude Oil	2024	75.76	5.18	86.91000366210938	65.75
	Crude Oil	2025	64.74	5.28	80.04000091552734	55.27000045776367
	Crude Oil	2026	61.37	2.6	65.41999816894531	55.9900016784668
	Natural Gas	2023	2.61	0.33	3.575000047683716	1.991000056266785

Explanation:

This query summarizes yearly performance for Crude Oil and Natural Gas, showing average, volatility, maximum, and minimum closing prices for historical trend and risk analysis.

Compare Crude vs Gas:

Purpose:

1. **Daily Price Comparison:** Shows day-by-day relative performance between Crude Oil and Natural Gas.
2. **Market Insight:** Helps identify correlations, gaps, or divergences in daily closing prices.
3. **Analysis & Reporting:** Useful for traders and analysts to compare commodity behaviour in the same period.

```
94  -- Compare Crude vs Gas
95  • SELECT c.Date,c.Close AS Crude_Close,g.Close AS Gas_Close,
96  ROUND(c.Close - g.Close, 2) AS Close_Difference
97  FROM commodity_db.crude_natural_gas c
98  JOIN commodity_db.crude_natural_gas g
99  ON c.Date = g.Date
100 WHERE c.Commodity = 'Crude Oil' AND g.Commodity = 'Natural Gas';
---
```

Output:

Result Grid				
Filter Rows:		Export: Wrap Cell Content:		
	Date	Crude_Close	Gas_Close	Close_Difference
▶	2023-02-15	78.58999633789062	2.470999956130981	76.12
	2023-02-16	78.48999786376953	2.388999938964844	76.1
	2023-02-17	76.33999633789062	2.275000095367432	74.06
	2023-02-21	76.16000366210938	2.072999954223633	74.09
	2023-02-22	73.94999694824219	2.174000024795532	71.78

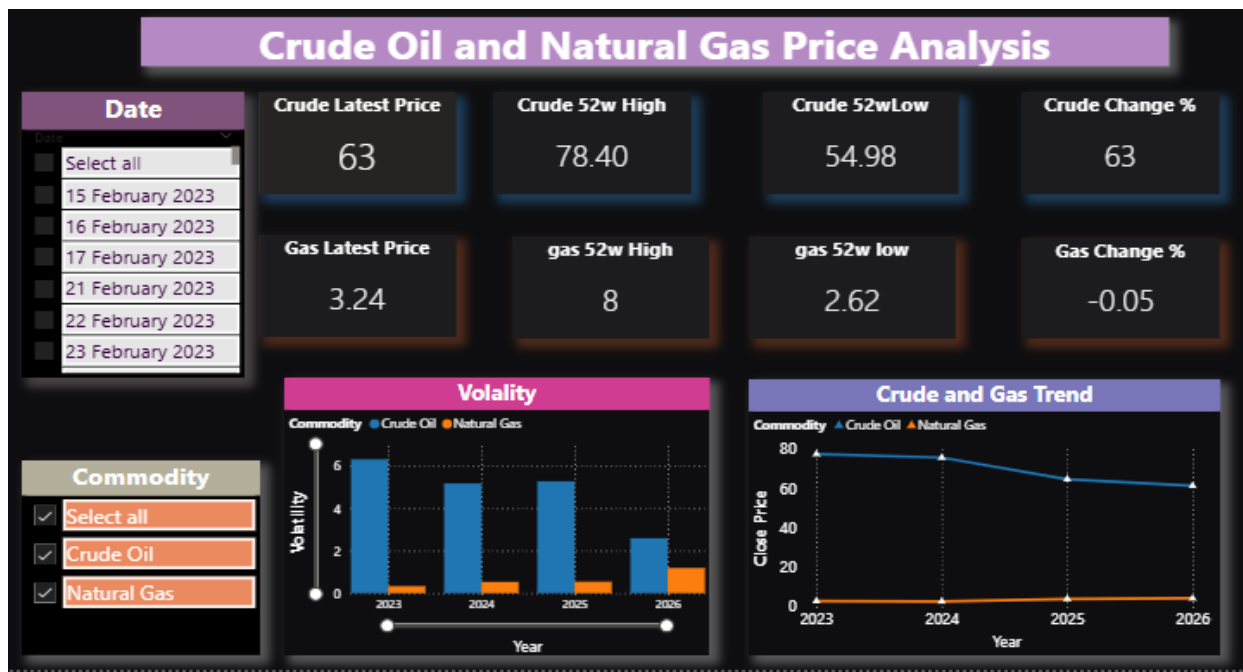
Explanation

This query compares daily closing prices of Crude Oil and Natural Gas, including the absolute difference, to analyse relative performance and market correlation.

POWERBI

Purpose:

The purpose of this dashboard is to provide a **comprehensive analysis of crude oil and natural gas prices**. It enables users to track market movements, compare historical highs and lows, monitor volatility, and identify long-term trends. This supports **business decision-making, investment strategies, and risk management** in the energy sector.



Dashboard Components:

Date Selector:

Allows filtering by specific dates, enabling time-based comparisons.

Commodity Selector:

Let the user choose between crude oil and natural gas, or view both together.

Latest Price:

Displays the current market price of each commodity, useful for tracking immediate value.

➤ DAX Function:

```
1 Crude Latest Price =
2 CALCULATE(
3     LASTNONBLANKVALUE(
4         crude_natural_gas[Date],
5         AVERAGE(crude_natural_gas[Close])
6     ),
7     crude_natural_gas[Commodity] = "Crude Oil"
8 )
```

```
1 Gas Latest Price =
2 CALCULATE(
3     LASTNONBLANKVALUE(
4         crude_natural_gas[Date],
5         AVERAGE(crude_natural_gas[Close])
6     ),
7     crude_natural_gas[Commodity] = "Natural Gas"
8 )
```


52-Week High/Low:

Shows the highest and lowest prices in the past year, helping identify trading ranges and market extremes.

Dax Function:

```
1 Crude 52W High =
2 VAR MaxDate =
3     CALCULATE(
4         MAX(crude_natural_gas[Date]),
5         crude_natural_gas[Commodity] = "Crude Oil"
6     )
7 RETURN
8 CALCULATE(
9     MAX(crude_natural_gas[High]),
10    crude_natural_gas[Commodity] = "Crude Oil",
11    crude_natural_gas[Date] >= MaxDate - 365
12 )
```

```
1 Crude 52W Low =
2 VAR MaxDate =
3     CALCULATE(
4         MAX(crude_natural_gas[Date]),
5         crude_natural_gas[Commodity] = "Crude Oil"
6     )
7 RETURN
8 CALCULATE(
9     MIN(crude_natural_gas[Low]),
10    crude_natural_gas[Commodity] = "Crude Oil",
11    crude_natural_gas[Date] >= MaxDate - 365
12 )
```

```
1 Gas 52W High =
2 VAR MaxDate =
3     CALCULATE(
4         MAX(crude_natural_gas[Date]),
5         crude_natural_gas[Commodity] = "Natural Gas"
6     )
7 RETURN
8 CALCULATE(
9     MAX(crude_natural_gas[High]),
10    crude_natural_gas[Commodity] = "Natural Gas",
11    crude_natural_gas[Date] >= MaxDate - 365
12 )
```

```

1 Gas 52W Low =
2 VAR MaxDate =
3     CALCULATE(
4         MAX(crude_natural_gas[Date]),
5         crude_natural_gas[Commodity] = "Natural Gas"
6     )
7 RETURN
8 CALCULATE(
9     MIN(crude_natural_gas[Low]),
10    crude_natural_gas[Commodity] = "Natural Gas",
11    crude_natural_gas[Date] >= MaxDate - 365
12 )

```

Change Percentage:

Highlights short-term momentum, showing whether prices are rising or falling.

Dax Function:

```

Crude Change % =
CALCULATE(
    AVERAGE(crude_natural_gas[Change%]),
    crude_natural_gas[Commodity] = "Crude Oil"
)

1 Gas Change % =
2 CALCULATE(
3     AVERAGE(crude_natural_gas[Change%]),
4     crude_natural_gas[Commodity] = "Natural Gas"
5 )
6

```

Volatility Chart:

Presents yearly volatility levels, which are critical for assessing risk and uncertainty in commodity markets.

Dax Function:

```

1 Volatility =
2 STDEVX.P (
3     VALUES ( 'crude_natural_gas'[Date] ),
4     'crude_natural_gas'[Close Price]
5 )

```

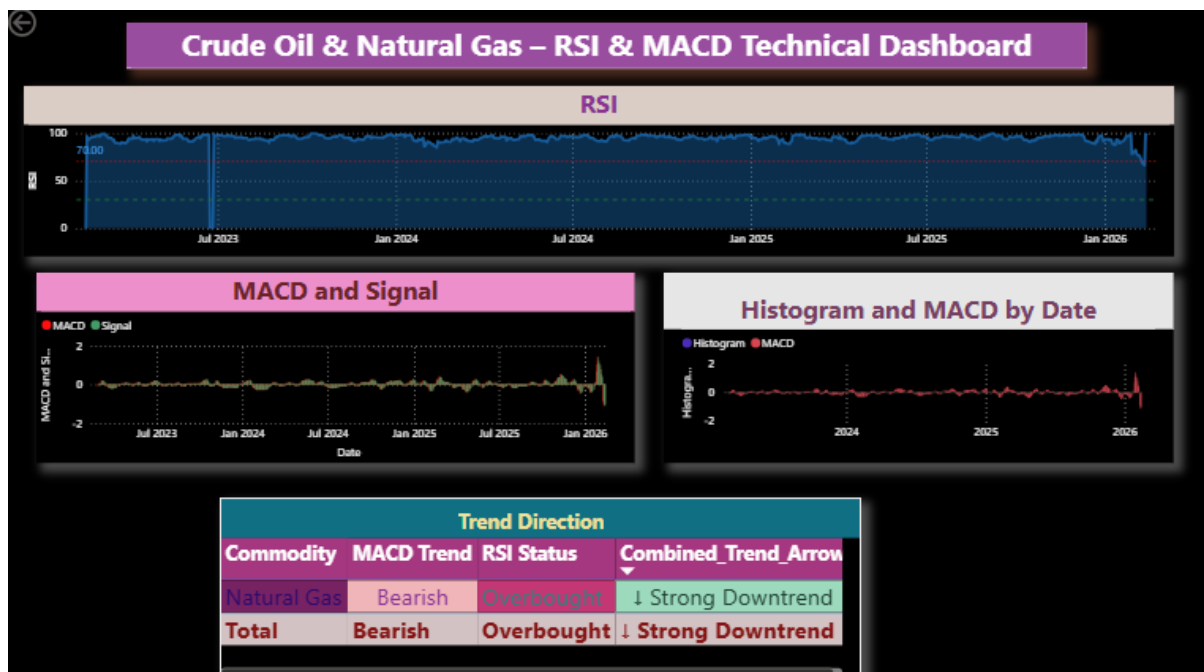
Trend Line Chart:

Illustrates long-term price movements, revealing whether the market is in decline, growth, or stability.

Insights

- **Crude Oil:** Shows moderate decline over years, with higher volatility in 2024–2025.
- **Natural Gas:** Displays sharper decline, with volatility spikes in 2023 and 2025.
- **Comparison:** Crude oil is relatively more stable than natural gas, but both show downward trends.

Technical Indicators:



RSI (Relative Strength Index)

- **Purpose:** Measures momentum — how fast and strong price movements are.
- **Scale:** 0 to 100.
 - **Above 70: Overbought** → price may be too high, possible reversal or pullback.
 - **Below 30: Oversold** → price may be too low, potential bounce or recovery.

DAX Function:

AVG Gain

AVG Loss

```

1 Avg Gain =
2 AVERAGEX(
3   DATESINPERIOD(
4     'crude_natural_gas'[Date],
5     MAX('crude_natural_gas'[Date]),
6     -14,
7     DAY
8   ),
9   [Gain]
10 )
11

```

```

1 Avg Loss =
2 AVERAGEX(
3   DATESINPERIOD(
4     'crude_natural_gas'[Date],
5     MAX('crude_natural_gas'[Date]),
6     -14,
7     DAY
8   ),
9   [Loss]
10 )
11

```

RSI:

```

1 RSI =
2 VAR RS = DIVIDE([Avg Gain], [Avg Loss])
3 RETURN
4 100 - (100 / (1 + RS))
5

```

RSI Status:

```

1 RSI Status =
2 IF([RSI] > 70, "Overbought",
3 IF([RSI] < 30, "Oversold", "Neutral"))
4

```

MACD (Moving Average Convergence Divergence)

- **Purpose:** Detects trend direction and momentum shifts.
- **Components:**
 - **MACD Line:** Difference between 12-day and 26-day EMAs.
 - **Signal Line:** 9-day EMA of the MACD line.
 - **Histogram:** Visual difference between MACD and Signal — shows strength of momentum.

DAX Function:

EMA12

EMA26

```

1 EMA 12 =
2 CALCULATE(
3     [Close Price],
4     DATESINPERIOD(
5         'crude_natural_gas'[Date],
6         MAX('crude_natural_gas'[Date]),
7         -12,
8         DAY
9     )

```

```

1 EMA 26 =
2 CALCULATE(
3     [Close Price],
4     DATESINPERIOD(
5         'crude_natural_gas'[Date],
6         MAX('crude_natural_gas'[Date]),
7         -26,
8         DAY
9     )

```

MACD

```

1 MACD =
2 [EMA 12] - [EMA 26]
3

```

Signals:

- **MACD crosses above Signal:** Bullish momentum.
- **MACD crosses below Signal:** Bearish momentum.

```

1 Signal =
2 CALCULATE(
3     [MACD],
4     DATESINPERIOD(
5         'crude_natural_gas'[Date],
6         MAX('crude_natural_gas'[Date]),
7         -9,
8         DAY
9     )
10

```

MACD Trend

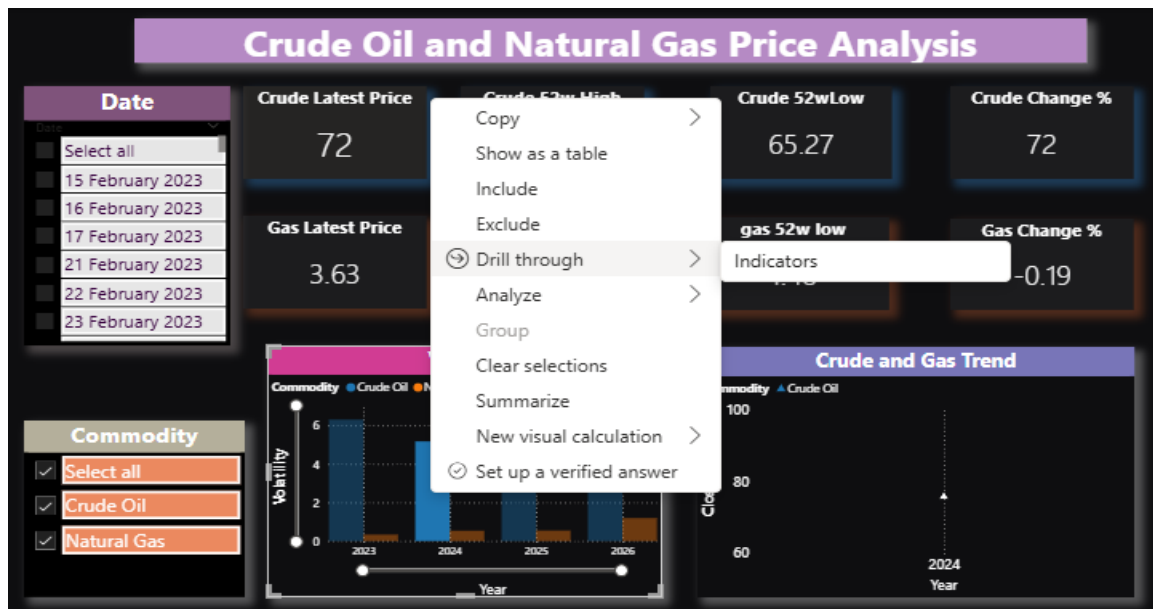
```

1 MACD Trend =
2 IF([MACD] > [Signal], "Bullish", "Bearish")
3

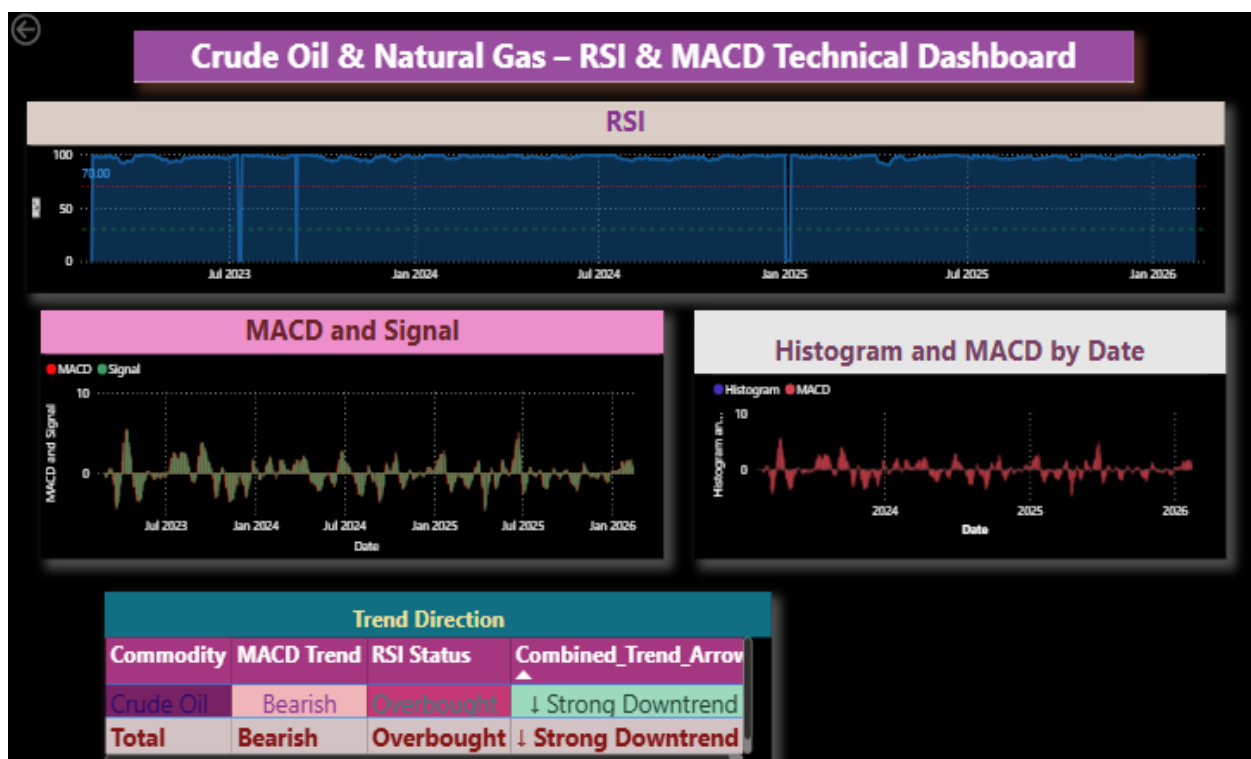
```

Drill Through- Indicators – Crude Oil

The Drill Through feature in the dashboard allows users to move from a **summary view** into a **detailed analysis page**. This enhances interactivity and provides deeper insights into specific commodities and time periods



When a user selects Crude Oil volatility from the bar chart (for example, the year 2024) and applies the Drill Through → Indicators option, the dashboard opens a dedicated page showing detailed indicators for Crude Oil.



RSI Chart (Top)

- RSI values fluctuate over time. For crude oil, RSI is currently **above 70**, signalling an **overbought** condition — a potential warning for a price pullback

MACD and Signal Chart (Bottom Left)”

- Crude oil’s MACD is **below the signal line**, indicating **bearish momentum**.

Histogram:

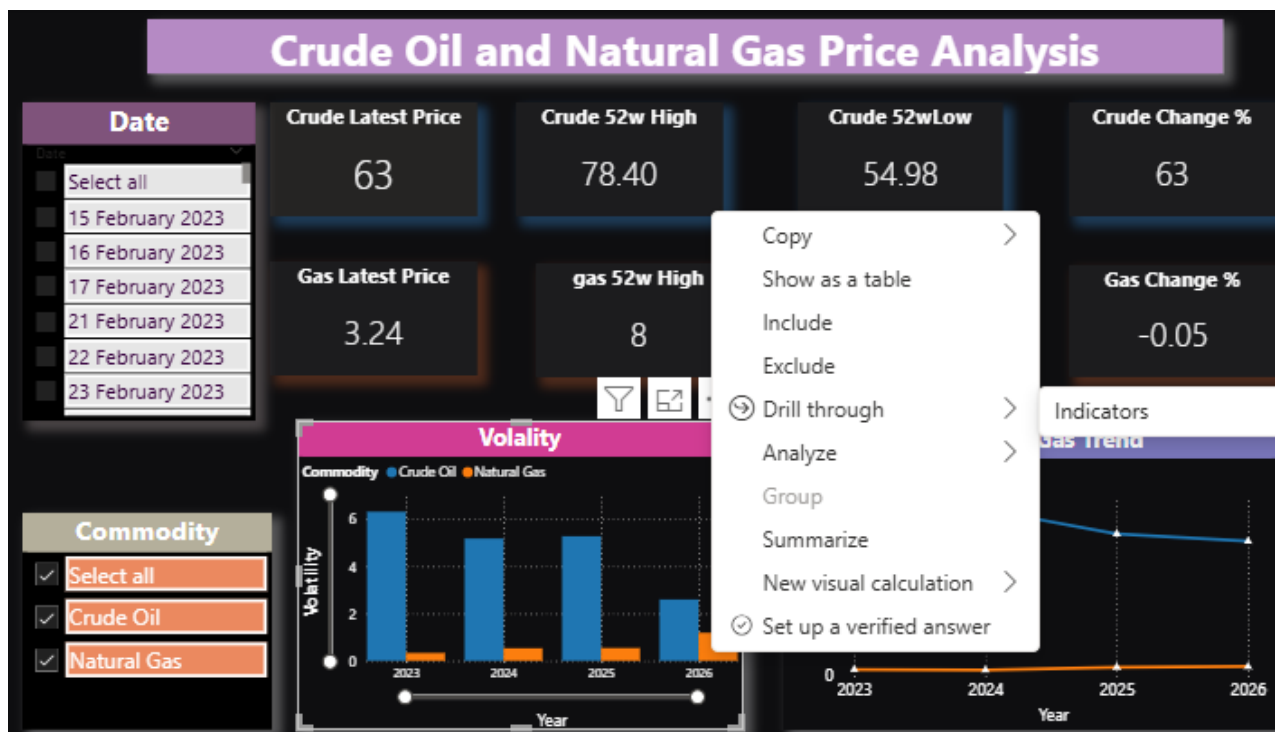
- Histogram bars are mostly **negative**, confirming **weakening momentum** in crude oil.

Trend Direction Table (Bottom Centre)

- Commodity: Crude Oil
- MACD Trend: Bearish
- RSI Status: Overbought
- Combined Trend Arrow: ↓ Strong Downtrend

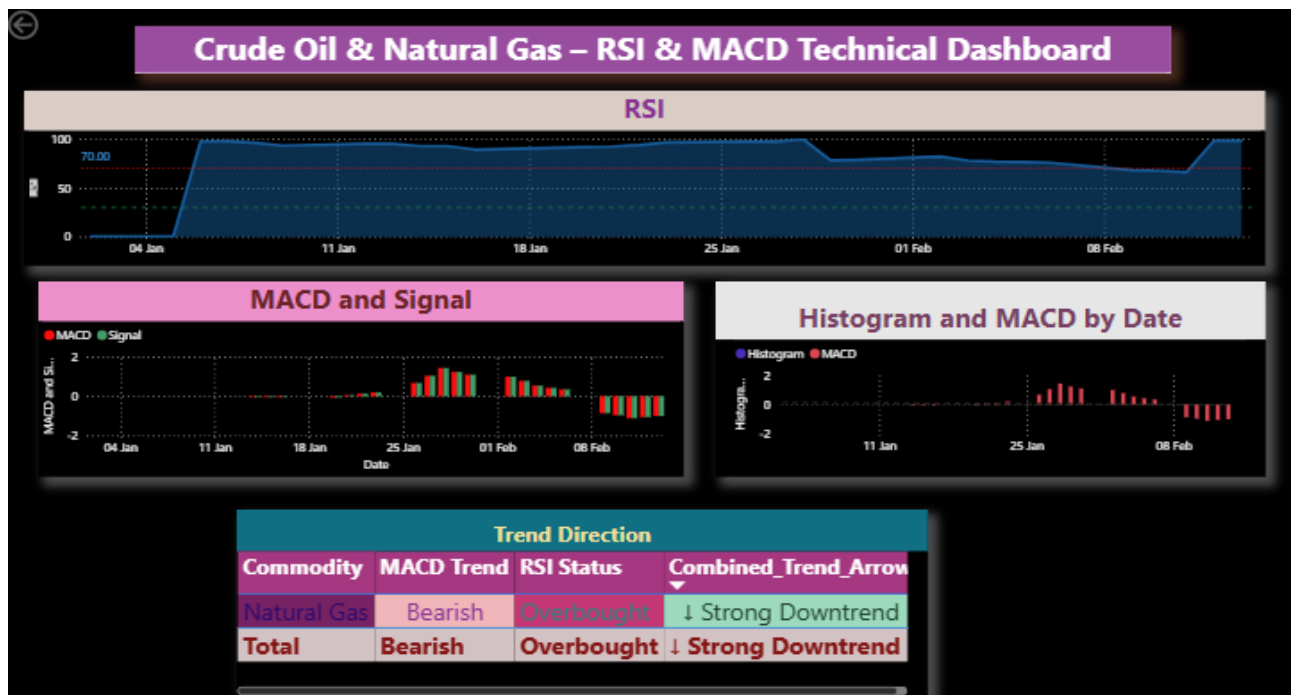
This means although the price is high (overbought), momentum is turning negative — a strong signal for a potential **price reversal or correction**.

Drill Through Indicators – Natural Gas (2026)



Explanation:

This Drill Through page provides a **detailed technical and trend analysis** of Natural Gas for the year 2026. While the main dashboard shows overall price and volatility, this page focuses on **specific indicators and signals** to understand market behaviour in depth.



Key Indicators

- **Relative Strength Index (RSI)**
 - RSI crossed above 70 in early February, signalling **overbought** conditions for natural gas.
- **MACD (Moving Average Convergence Divergence)**
 - Compares short-term and long-term moving averages.
 - Current MACD trend is **bearish**, signalling downward momentum.
 - Crossovers between MACD and the signal line confirm sell signals.
- **Histogram**
 - Shows the difference between MACD and the signal line.
 - Red bars below zero highlight **negative momentum**, reinforcing the bearish outlook.
- **Trend Direction Summary**
 - Commodity: Natural Gas
 - MACD Trend: Bearish
 - RSI Status: Overbought
 - Combined Trend: ↓ Strong Downtrend

This combination suggests that although prices are high (overbought), momentum is turning negative — a strong warning for a potential price drop.

Business Impact

- **Traders:** Entry/Exit timing
- **Investors:** Risk assessment
- **Energy Companies:** Price forecasting
- **Policy Makers:** Supply-demand planning.

Conclusion

This project successfully demonstrates how **data analytics combined with technical indicators and global news events** can explain price movements in energy commodities.

- Crude oil is **highly sensitive to geopolitical and macroeconomic events**
- Natural gas is **generally stable but prone to sudden spikes**
- Technical indicators (RSI, MACD, MA) effectively identify risk and trend reversals

Overall, the project highlights the importance of **data-driven decision-making in energy markets** and showcases strong analytical and business interpretation skills.

Future Scope:

- Price Forecasting using Machine Learning.
- Real-Time Market Data Integration.
- News & Sentiment Analysis (NLP)
- Advanced Risk Metrics (Va R, CVaR, Drawdown)
- Multi-Commodity Expansion
- Automated Trading Strategy & Back testing.
- Cloud & Big Data Integration
- Advanced Power BI Enhancements
- AI-Driven Market Intelligence System